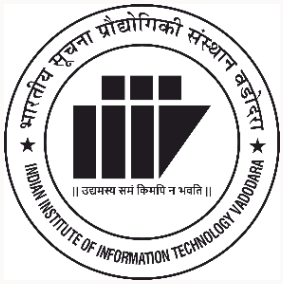
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
VADODARA



CS545 | SOFTWARE ENGINEERING

PROJECT REPORT

Master of Computer Applications (MCA) | Second Year

DISTRIBUTED SYSTEMS | TRANSACTIONAL STORAGE

kvstore

A Distributed, Fault-Tolerant, Transactional Key–Value Store

with an Integrated Operational Intelligence Layer

GROUP NO. 1

SUBMITTED BY

Neha Nupur (Team Leader)
20251651064

Aditi Garg (Member)
20251651008

Shivam Kumar Soni (Member)
20251651084

SUBMITTED TO

Rishabh Gupta
Faculty | Department of CS/IT

AUTUMN 2026–27

Certificate

This is to certify that the project and laboratory work documented in this report for CS545 – Software Engineering has been prepared by Group 1, students of the Master of Computer Applications programme at the Indian Institute of Information Technology Vadodara, during the academic year 2026–27. The report covers the requirements, analysis, modelling, design, modularisation, and test-design activities for the *kvstore* project.

S. No.	Student Name	Enrollment No.
1	Neha Nupur	20251651064
2	Aditi Garg	20251651008
3	Shivam Kumar Soni	20251651084

Date: _____ **Faculty Signature:** _____

Faculty Name: Rishabh Gupta

Declaration

We, the undersigned students, declare that this report documents the work prepared by Group 1 for the CS545 Software Engineering laboratory. The requirements, analyses, diagrams, design descriptions, and test cases are presented for academic assessment. External materials referenced in preparing the report are listed in the References section.

Student	Enrollment No.	Signature
Neha Nupur	20251651064	_____
Aditi Garg	20251651008	_____
Shivam Kumar Soni	20251651084	_____

Date: _____

Acknowledgement

We would like to express our sincere gratitude to Mr. Rishabh Gupta, our course faculty, for his guidance and feedback throughout the CS545 Software Engineering laboratory. His direction helped us approach the requirements, modelling, architectural design, and testing activities in a structured manner. We are also thankful to the Indian Institute of Information Technology Vadodara for providing the academic environment and resources needed to complete this work. Finally, we acknowledge the collaboration and shared effort of every member of Group 1 in developing and documenting the *kvstore* project. *Group 1 | CS545 Software Engineering | Autumn 2026–27*

Contents

Certificate i

Declaration ii

Acknowledgement iii

Experiment Index vii

1 Introduction and Product Overview 1

1.1 Purpose . . . 1

1.2 Problem Statement . . . 1

1.3 Goals . . . 1

1.4 Scope and Boundaries . . . 1

1.4.1 In scope . . . 1

1.4.2 Out of scope or externally dependent . . . 2

1.5 Stakeholders . . . 2

2 System Context and High-Level Architecture 3

2.1 External Entities . . . 3

2.2 Context and Data-Flow Views . . . 3

2.3 Subsystem Overview . . . 3

2.4 Internal Package Architecture . . . 4

3 Requirements Specification 5

3.1 Functional Requirements . . . 5

3.2 Non-Functional Requirements . . . 11

3.3 Constraints and Assumptions . . . 11

Laboratory Record: Experiment 1 – Software Requirements Specification (SRS) . . . 11

4 Software Process Model and Development Plan 15

4.1 Selection Rationale . . . 15

4.2 Chosen Approach: Risk-Driven Iterative Incremental Development . . . 15

4.3 Risk Register . . . 15

4.4 Case Analyses from the Lab 2 Handout . . . 16

4.4.1 Case 1: College Payroll System – Waterfall . . . 16

4.4.2 Case 2: New Food-Delivery App – Agile . . . 16

4.4.3 Case 3: Hospital Emergency Monitoring System – Spiral . . . 16

4.4.4 Case 4: New College Portal Interface – Prototyping . . . 17

4.5 Decision Factors and Weighted Decision Matrix . . . 17

4.6 Practical Process Controls . . . 17

4.7 Development Increments . . . 17

Laboratory Record: Experiment 2 – Selecting a Software Process Model . . . 18

5 Use-Case Model 20

5.1 Use-Case Diagram . . . 20

5.2 Actors . . . 20

5.3 Principal Use-Case Descriptions . . . 21

5.4 Use-Case Relationships . . . 21
Laboratory Record: Experiment 3 – Use Case Diagram . . . 21
6 Data-Flow Modelling 23
6.1 Level 0: Context Diagram . . . 23
6.2 Level 1: Subsystem Data Flow . . . 23
6.3 Level 2: Internal Package Architecture . . . 24
6.4 Level 2 Internal Package Interpretation . . . 25
6.4.1 Named internal flows . . . 25
6.5 Balancing and Consistency Check . . . 26
Laboratory Record: Experiment 4 – Data Flow Diagram (DFD) . . . 26
7 UML Structural Design 29
7.1 Class Diagram . . . 29
7.2 Important Classes and Responsibilities . . . 29
7.3 Design Principles Reflected . . . 29
Laboratory Record: Experiment 5 – UML Class Diagram . . . 30
8 Interaction and Sequence Design 33
8.1 Scenario 1: PUT with Leader Redirect . . . 33
8.2 Scenario 2: Transaction with Snapshot Isolation . . . 33
8.3 Scenario 3: Cross-Shard Two-Phase Commit . . . 33
8.4 Cross-Scenario Invariants . . . 33
Laboratory Record: Experiment 6 – Sequence Diagrams . . . 34
9 Modular Design 38
9.1 Design Objectives . . . 38
9.2 Module Architecture . . . 38
9.3 Module Responsibilities and Interfaces . . . 38
9.4 Cohesion and Coupling Review . . . 38
9.5 Architecture Decisions . . . 39
Laboratory Record: Experiment 8 – Modular Design of a Software System . . . 39
9.6 Project Requirements Used for Modularisation . . . 42
9.7 Responsibility Inventory . . . 42
9.8 Thirteen Modules and Package Mapping . . . 43
9.9 Module Interfaces . . . 43
9.9.1 Interface 1: Transaction API . . . 43
9.9.2 Interface 2: Wire Envelope . . . 44
9.9.3 Interface 3: StatsProvider . . . 44
9.9.4 Interface 4: Key Routing . . . 44
9.10 Cohesion and Coupling Decisions . . . 44
9.11 Coupling Numericals from Lab 8 . . . 45
9.12 Module-to-Code Traceability from Lab 8 . . . 46
10 Implementation Overview 47
10.1 Technology and Repository View . . . 47
10.2 Key Implementation Areas . . . 47
10.3 Error Handling Principles . . . 47
11 Functional Test Strategy and Verification 48
11.1 Testing Objectives . . . 48
11.2 Test Levels . . . 48
11.3 Test Register . . . 48
11.4 Detailed Test Procedure: PUT and GET . . . 49

11.5 Detailed Test Procedure: MVCC Conflict . . . 50
11.6 Detailed Test Procedure: Crash and Restart Recovery . . . 50
11.7 Detailed Test Procedure: Cross-Shard Atomicity . . . 50
11.8 Traceability Summary . . . 51
11.9 Implementation Evidence and Status Discipline . . . 51
Laboratory Record: Experiment 9 – Functional Test Case Design . . . 51
11.10 Selected Lab 9 Requirements . . . 53
11.11 Original Register A1: Scenario and Classification . . . 54
11.12 Test Procedure Details and Pass Rules . . . 54
11.12.1 TC-01: Create a key and read it back . . . 54
11.12.2 TC-06: Conflicting MVCC writes . . . 55
11.12.3 TC-07: Crash and restart recovery . . . 55
11.12.4 TC-08: Request through a follower . . . 55
11.12.5 TC-10: Prometheus metrics . . . 55
11.12.6 TC-11: Unsupported operational query . . . 55
11.12.7 TC-12: Untrusted TLS certificate . . . 55
11.12.8 TC-14 and TC-15: Cross-shard transaction boundary . . . 55
11.13 Repository Test Evidence Named in Lab 9 . . . 55
11.14 Traceability and Execution Record . . . 56

12 Security, Reliability, and Operational Considerations 57
12.1 Security . . . 57
12.2 Reliability and Recovery . . . 57
12.3 Operational Monitoring . . . 57

13 Limitations and Future Enhancements 58
13.1 Current Documentation Limitations . . . 58
13.2 Recommended Enhancements . . . 58

14 Conclusion 59

A Consolidation Map: Laboratory Artefacts 60

B Quick Reference: Key Terms 61

C Team and Submission Information 62

References 63

Experiment Index

This index provides direct navigation to the laboratory experiments documented in this report. Page numbers are generated automatically.

Experiment	Title	Page
1	Software Requirements Specification (SRS)	11
2	Selecting a Software Process Model	18
3	Use Case Diagram	21
4	Data Flow Diagram (DFD)	26
5	UML Class Diagram	30
6	Sequence Diagrams	33
8	Modular Design of a Software System	39
9	Functional Test Case Design	51

The supplied laboratory report does not contain substantive content for Experiment 7; no experiment has been invented.

Introduction and Product Overview

1.1 Purpose

The purpose of kvstore is to provide a reliable key–value service that can continue to operate in a distributed cluster while maintaining a consistent committed state. In addition to core data operations, the system includes administrative observability to help an operator understand cluster behaviour and investigate anomalies.

1.2 Problem Statement

A single-process in-memory key–value map is easy to implement but does not by itself provide replicated durability, leadership handling, transaction isolation, recovery after a crash, shard-aware routing, or operational diagnosis. kvstore addresses these concerns through distinct subsystems and explicit protocols. The system must also make failures visible through clear API outcomes rather than silently applying partial or conflicting writes.

1.3 Goals

- Provide create/update, retrieve, and delete operations for key–value entries.
- Route writes through the correct shard and active Raft leader.
- Replicate log entries and commit them only when the required consensus conditions are met.
- Preserve committed writes through durable logging and restart recovery.
- Support transactions with snapshot visibility and conflict detection.
- Coordinate multi-shard transactions with a durable two-phase-commit decision record.
- Expose metrics and operational diagnostics while keeping the AI provider outside the data-commit trust boundary.
- Validate operational endpoints, TLS trust, and restricted natural-language queries.

1.4 Scope and Boundaries

1.4.1 In scope

Core key–value operations; shard routing; Raft leader and follower behaviour; WAL-backed recovery; MVCC and HLC; transaction commit/abort; cross-shard coordination; REST/JSON interfaces; metrics; TLS validation; and bounded AI-assisted operational queries.

1.4.2 Out of scope or externally dependent

The external LLM is not a database authority and must not decide whether a transaction commits. Its output is advisory. Production deployment topology, authentication provider configuration, performance service-level objectives, and operational retention settings depend on the selected deployment and must be confirmed before production use.

1.5 Stakeholders

Stakeholder Interest / responsibility

Client application Submits key–value and transaction requests; consumes results and errors.

Cluster operator / Monitors health, reviews metrics and alerts, manages nodes and certificates,

administrator and investigates incidents.

System Configures deployment, security, and operational policy.

administrator

External LLM Receives bounded diagnostic evidence and returns hypotheses; it is not

provider trusted with commit authority.

Developer / tester Maintains modules, validates requirements, and records repeatable test evidence.

System Context and High-Level Architecture

2.1 External Entities

Table 2.1: *External entities and system exchanges*

Entity	Input to kvstore	Output from kvstore
Client application	PUT/GET/DELETE operations, transaction commands, request metadata.	Values, acknowledgements, transaction outcomes, redirects, and error responses.
Cluster operator	Dashboard and metrics queries, operational questions, cluster-management actions.	Live dashboard, performance measures, alerts, diagnostic insights, and configuration outcomes.
External LLM provider	Receives anomaly evidence / diagnostic context from the observability layer.	Returns root-cause hypotheses or explanations; it is not trusted with commit authority.

2.2 Context and Data-Flow Views

The context, Level 1, and Level 2 diagrams are presented together in the dedicated Data-Flow Modelling chapter, each as a separate portrait page. This separation is intentional: it preserves the original diagram images and labels, avoids stacking different abstraction levels into one figure, and gives each diagram its own interpretation and data-flow inventory. Figure 6.1 defines the external boundary; Figure 6.2 decomposes the system into major processes; Figure 6.3 expands those processes into internal packages.

2.3 Subsystem Overview

Table 2.2: *High-level subsystem responsibilities*

Subsystem	Primary responsibility	Principal information handled
1.0 API and Routing	Accepts requests, validates input, routes by key/shard, and handles leader redirects.	Requests, route information, client outcomes.
2.0 Consensus	Uses Raft to elect leaders, replicate proposals, and determine commitment.	Proposals, acknowledgements, committed entries.
3.0 Storage	Applies committed entries and manages durable state, HLC, and MVCC.	Committed entries, versions, WAL records.
4.0 Observability	Collects performance signals, serves metrics, and produces diagnostic insights.	Metric streams, alerts, anomaly evidence.

2.4 Internal Package Architecture

The Level 2 view expands the subsystems into packages: HTTP API/transport, client library, shard routing, Raft, WAL, storage, HLC, MVCC, metrics, and the operational intelligence engine. The full supplied diagram and its package-level interpretation appear in Chapter 6. The package view makes responsibilities explicit and supports testing individual modules as well as end-to-end flows.

Requirements Specification

3.1 Functional Requirements

The formal Software Requirements Specification defines 84 functional requirements, identified as SRS FR 001 through SRS FR 084. The complete requirement statements are listed below with their original identifiers. The wording is retained from the supplied SRS text; the table is formatted for readability and traceability.

SRS ID	Functional requirement statement
FR 001	The system shall generate, for every event requiring a timestamp, a composite value consisting of a physical wall clock reading, a logical counter, and the identifier of the generating node.
FR 002	The system shall increase the logical counter component of a generated timestamp whenever the physical wall clock reading has not advanced relative to the previous timestamp generated by the same node.
FR 003	The system shall provide a comparison function over generated timestamps that orders first by physical component, second by logical component, and third by node identifier, yielding a strict total order across the entire cluster.
FR 004	The system shall provide a mechanism for folding a timestamp received from a remote node into the local clock state such that causal ordering between the local node and the remote node is preserved when that mechanism is invoked.
FR 005	The system shall support a configurable maximum tolerated clock drift and shall support the artificial injection of a clock offset for the purpose of automated testing of clock skew handling.
FR 006	The system shall document, as a known and explicit limitation rather than as an implicit assumption, that no message currently transmitted over the network protocol carries a remote timestamp for automatic folding during ordinary production operation, and that cross-node ordering during ordinary operation instead depends on the node identifier component of every timestamp together with each node's physical clock remaining within the configured maximum tolerated drift of its peers.
FR 007	The system shall append an arbitrary byte sequence record to a log file together with a checksum computed over the contents of that record.
FR 008	The system shall force a physical flush of every appended record to the underlying storage device before reporting that record as durably persisted.
FR 009	The system shall, during replay of a log file after a restart, recompute the checksum of every stored record and shall treat the first record whose recomputed checksum does not match its stored checksum, or the first record that is truncated, as the effective end of the log, discarding nothing that precedes it and trusting nothing beyond it.
FR 010	The system shall use this write ahead log mechanism to persist every Raft log entry, the current term and vote record, and every periodic snapshot for each cluster node.
FR 011	The system shall maintain, for every cluster node at every moment, exactly one of the three roles leader, follower, or candidate.
FR 012	The system shall elect at most one leader per Raft term across the entire cluster, verified specifically under contested, near simultaneous election conditions.

Continued on the next page

SRS ID	Functional requirement statement (continued)
FR 013	The system shall use a randomised election timeout, configurable through a minimum and maximum bound, so that followers do not simultaneously become candidates.
FR 014	The system shall require that a candidate's replicated log be at least as up to date as a voting node's own log before that voting node grants its vote, and shall permit a voting node to grant at most one vote per term.
FR 015	The system shall replicate every operation accepted by the leader to every follower and shall consider that operation committed only once a majority of the cluster, including the leader itself, has durably stored it.
FR 016	The system shall detect the point at which a follower's log diverges from the leader's log and shall bring that follower back into agreement using an accelerated conflict back off mechanism rather than retransmitting one entry at a time.
FR 017	The system shall periodically compact its log through snapshotting and shall discard log entries made redundant by a completed snapshot.
FR 018	The system shall transfer a complete snapshot directly to any follower that has fallen so far behind that the leader has already discarded the log entries that follower requires, after which that follower shall resume ordinary incremental replication.
FR 019	The system shall support live changes to cluster membership, including the addition and removal of voting members, using a joint consensus configuration that requires majority agreement from both the old and the new membership lists during the transitional period.
FR 020	The system shall automatically propose the finalising configuration entry, dropping the old membership list, once the joint configuration entry has itself been committed.
FR 021	The system shall support a deliberate, voluntary leadership transfer from the current leader to a specified follower, ensuring the target follower's log is fully caught up before instructing it to begin an election immediately, without waiting for its normal election timeout.
FR 022	The system shall never permit a network partition to allow a minority subset of the cluster to elect a leader or to commit a write, ensuring that a full network partition can never result in the two disjoint sides of the cluster independently and silently diverging into inconsistent data.
FR 023	The system shall store, for every key, an ordered chain of versions, each version tagged with the hybrid logical clock timestamp at which it was written.
FR 024	The system shall assign every new transaction a beginning timestamp representing the instant the transaction started.
FR 025	The system shall ensure that every read performed by a transaction, for the remainder of that transaction's lifetime, returns the version of the requested key that was most recently committed as of that transaction's beginning timestamp, regardless of any transaction that commits later while this transaction is still active.
FR 026	The system shall reject, at commit time, any transaction for which some key it wrote has had a newer version committed since that transaction's beginning timestamp, applying a first committer wins conflict resolution rule.
FR 027	The system shall perform the authoritative conflict check for a committing transaction deterministically, at the moment that transaction's write set is applied from the Raft log on every replica, so that every replica independently reaches the identical commit or abort decision.
FR 028	The system shall additionally perform an optimistic, non authoritative conflict check before a transaction is proposed through Raft at all, so that a transaction that is already certain to fail can be rejected quickly without incurring the cost of a full consensus round trip.

Continued on the next page

SRS ID	Functional requirement statement (continued)
FR 029	The system shall assign every transaction a unique identifier derived deterministically from that transaction's hybrid logical clock beginning timestamp tuple, ensuring uniqueness across the whole cluster and across process restarts.
FR 030	The system shall reclaim, through a garbage collection process, any version of a key older than the beginning timestamp of the oldest still active transaction in the cluster, while never reclaiming the single most recent remaining version of any key.
FR 031	The system shall support a temporary, exclusive intent lock on a set of keys on behalf of a transaction that has been tentatively prepared but not yet resolved, during which no ordinary read is permitted to observe the locked keys in an uncommitted state.
FR 032	The system shall expose a transactional application programming interface supporting the operations begin, get, put, delete, and commit, callable by code elsewhere in the system.
FR 033	The system shall, upon a commit request, propose the transaction's complete write set through Raft as a single log entry and shall wait for that entry to be safely and deterministically applied to a majority of the cluster before reporting success to the caller.
FR 034	The system shall function as the Raft state machine, meaning it shall receive every committed log entry from Raft in order, on every node, and shall interpret and apply that entry to the actual key–value data.
FR 035	The system shall encode every request as a single structured envelope message using a built in binary serialisation format and shall transmit it over a freshly opened connection for each individual request.
FR 036	The system shall dispatch both internal Raft protocol messages and external client key–value requests through the same server process and the same listening network address.
FR 037	The system shall, when a client request arrives at a node that is not the current leader, respond with a specific reply indicating that condition, and shall include, whenever known, the network address of the current leader as a hint.
FR 038	The system shall support an entirely optional Transport Layer Security encrypted mode for every connection, activated additively without changing default behaviour when disabled.
FR 039	The system shall support an optional mutual Transport Layer Security mode requiring every connecting client to present a certificate signed by a configured trusted authority.
FR 040	The system shall provide a client library capable of discovering which cluster node is currently the leader.
FR 041	The system shall automatically redirect a client request to the correct leader node when an initial request is answered with a not leader reply carrying a leader hint.
FR 042	The system shall retry a client request across multiple candidate cluster addresses when one or more addresses are temporarily unreachable, rather than failing immediately upon the first unreachable address.
FR 043	The system shall provide monotonically increasing counters, freely adjustable gauges, and histograms as its three fundamental instrumentation primitives.
FR 044	The system shall compute, for every histogram, the count, sum, minimum, maximum, running mean, and running standard deviation of recorded observations using a numerically stable, single pass algorithm.
FR 045	The system shall compute an exponentially weighted moving average for every histogram, emphasising the most recent observations over older ones.

Continued on the next page

SRS ID	Functional requirement statement (continued)
FR 046	The system shall compute approximate percentile estimates for every histogram using a fixed set of logarithmically sized buckets.
FR 047	The system shall automatically measure the latency and success or failure outcome of every client request handled by the network server, broken down by the specific operation performed.
FR 048	The system shall expose accumulated metrics data in both the standard Prometheus text exposition format and a plain JSON format.
FR 049	The system shall detect anomalies in a monitored metric series using two exponentially weighted moving averages tracked in parallel, one representing a slowly moving historical baseline and one representing rapidly moving current behaviour, and shall flag an anomaly when the current estimate diverges sufficiently from the baseline relative to that metric's own recent variability.
FR 050	The system shall compute the baseline variance estimate used in anomaly detection using a mechanism that is robust against being dragged upward by a sustained anomaly, so that a persistent problem does not gradually retrain the baseline to consider that problem normal.
FR 051	The system shall retain, for every tracked metric, its recent raw observation history in a bounded, fixed size rolling buffer, distinct from that metric's smoothed moving average values.
FR 052	The system shall, upon the detection of a critical anomaly in one metric, automatically compute the Pearson correlation coefficient between that metric and every other tracked metric over the same recent time window, and shall report the resulting correlated series together with the numerical strength of each correlation.
FR 053	The system shall fit a least squares regression line through a tracked metric's recent raw history and shall project that line forward to produce a trend forecast, reporting alongside that forecast a coefficient of determination indicating the goodness of fit of the underlying trend line.
FR 054	The system shall continuously score every node in the cluster on a combination of recently observed latency, error rate, replication lag, and queue depth.
FR 055	The system shall recommend a leadership transfer away from the current leader when a follower is clearly and consistently healthier by the node scoring mechanism, applying a hysteresis margin to prevent the recommendation from oscillating between two nodes of similar health.
FR 056	The system shall never allow the leadership advisor to directly instruct the consensus layer to perform a transfer; the advisor shall only ever produce a recommendation, with any actual transfer requiring either an explicit operator request or an explicit, separately configured automatic transfer setting.
FR 057	The system shall adaptively adjust the garbage collection interval and the automatic snapshotting threshold using a proportional feedback controller driven by the currently observed write rate and the degree of accumulated multi version concurrency control version bloat.
FR 058	The system shall answer common, plainly phrased natural language questions about cluster state, including questions about the number of stored keys, the identity of the current leader, recently detected anomalies, and a forecast for a named metric, using a fully deterministic, rule based parser requiring no external network dependency.

Continued on the next page

SRS ID	Functional requirement statement (continued)
FR 059	The system shall, only when an external large language model credential has been explicitly configured by the operator, fall back to a large language model for natural language questions the deterministic parser does not recognise, and shall constrain that large language model to selecting exclusively from among the same fixed, pre validated set of read only operations the deterministic parser itself supports, independently verifying every selection against that fixed list before executing it.
FR 060	The system shall, only when an external large language model credential has been explicitly configured, optionally hand already computed anomaly and correlation evidence to a large language model to obtain a plain language, prose only root cause hypothesis and suggested next step, and shall ensure this diagnosis capability never returns anything that is executed, runs asynchronously under a rate limit, and never blocks the system's own continuous monitoring loop.
FR 061	The system shall degrade gracefully and silently whenever the optional large language model call fails or is not configured, such that the deterministic anomaly and correlation insights already computed and recorded remain fully valid and available regardless.
FR 062	The system shall implement the entire operational intelligence layer without any direct dependency on the consensus, storage, or network transport code, obtaining all required cluster data exclusively through two narrowly defined interfaces supplied by an external adapter.
FR 063	The system shall generate a temporary, self signed certificate suitable for local development use when Transport Layer Security is enabled without an explicitly supplied certificate and key.
FR 064	The system shall load a real certificate and private key pair supplied by the operator for production deployment when explicitly configured to do so.
FR 065	The system shall support requiring every connecting client to present a certificate signed by a configured trusted authority when mutual Transport Layer Security is enabled.
FR 066	The system shall expose, on every node for which the Hypertext Transfer Protocol interface is enabled, a REST style application programming interface supporting reading, writing, and deleting a single key, submitting a multi key transaction, browsing the locally visible set of keys, retrieving node status, retrieving metrics in either supported format, and interacting with every capability of the operational intelligence layer.
FR 067	The system shall expose a continuous, live streaming feed of cluster events using the Server Sent Events web standard, chosen specifically because the required data flow is one directional, from server to browser, and because the standard provides automatic client side reconnection without additional implementation effort.
FR 068	The system shall serve a single, self contained, embedded browser dashboard requiring no separately deployed files and no externally fetched resources, presenting a live visual representation of the cluster arranged as nodes in a circle with visible connections to the current leader that pulse on each heartbeat, live charts of latency, throughput, and error rate, a browser for currently stored keys, a feed of operational intelligence insights, and a text input for submitting natural language questions.
FR 069	The system shall allow every node to optionally poll the status endpoint of every other configured peer node over ordinary Hypertext Transfer Protocol, building a complete, honestly self reported picture of cluster wide health for the leadership advisor, and shall ensure that a node with no peer polling configured limits itself to what it can observe locally and never recommends a leadership transfer in that configuration.

Continued on the next page

SRS ID	Functional requirement statement (continued)
FR 070	The system shall route each incoming key to exactly one of a configured set of independent shard clusters using rendezvous hashing, computing a deterministic numeric score for the key against every configured shard using the SHA two hundred fifty six hash function and selecting the shard with the highest score.
FR 071	The system shall ensure that the routing decision for any given key depends only on the fixed, configured set of shard identifiers and on the key itself, and never on any per process or per run randomised state, so that the identical routing decision is reached by any process constructing the routing table over the same shard set, including the same process across a restart.
FR 072	The system shall reject, before forwarding it to any shard, any ordinary multi key transaction whose keys span more than one shard, returning a clear error to the caller.
FR 073	The system shall provide a dedicated cross shard transaction endpoint supporting atomic, multi key writes and deletes across as many shards as the affected keys are routed to, implemented through a two phase commit coordinator.
FR 074	The system shall, as the first step of any cross shard transaction, durably record which shards that transaction will involve before contacting any of them.
FR 075	The system shall, during the prepare phase of a cross shard transaction, request that every involved shard lock its affected keys using the intent mechanism, aborting the entire transaction if any single shard declines to prepare.
FR 076	The system shall, once every involved shard has successfully agreed to prepare, durably record the final commit decision before instructing any shard to act upon it, this durable recording being the exact point at which the transaction's outcome becomes permanently fixed.
FR 077	The system shall instruct every involved shard to carry out the recorded commit or abort decision, retrying past transient failures until every shard has confirmed completion.
FR 078	The system shall, upon restart after a crash occurring during a cross shard transaction, replay its own durably recorded decision log and resolve any transaction for which no final decision was durably recorded before the crash as an abort.
FR 079	The system shall decline, with a clear explanatory error, any cross shard transaction request received by a gateway instance that was not configured with its own durable data directory.
FR 080	The system shall provide a server program, one running instance of which constitutes exactly one member of a Raft cluster, configurable through command line flags governing network addresses, data directory location, consensus timing, optional Transport Layer Security, and optional operational intelligence settings.
FR 081	The system shall provide a command line client program supporting reading, writing, deleting a key, and checking a node's status, built directly upon the shared client library.
FR 082	The system shall provide a sharding gateway program configured with the network addresses of each independent shard cluster it routes to, which shall function correctly for ordinary routed operations even without a configured data directory, declining only cross shard transaction requests in that configuration.
FR 083	The system shall detect, through a stored per record checksum, any corruption of a write ahead log record caused by a partial write during a crash, and shall treat the log as ending at the first such corrupted or truncated record during replay.
FR 084	The system shall ensure that every replica of the cluster applies the identical sequence of Raft log entries in the identical order, which is the property that guarantees every replica's multi version concurrency control state remains consistent with every other replica's state.

3.2 Non-Functional Requirements

The non-functional requirements describe the quality attributes and operational expectations that guide implementation and verification.

Table 3.2: *Non-functional requirements and quality expectations*

Quality attribute	Expectation
Consistency	Replicas should converge on the same committed log and deterministic state transitions.
Durability	Committed writes must survive a process crash when the configured durable storage guarantees hold.
Availability	The cluster should continue to accept eligible operations when a functioning leader and required quorum are available.
Security	TLS trust must be validated; administrative and operational surfaces must be constrained.
Maintainability	Modules should have high cohesion, low coupling, and explicit interfaces.
Observability	Metrics and diagnostic events should help operators understand latency, errors, and cluster state.
Recoverability	Startup recovery must reconstruct committed state and resolve durable transaction decisions.
Testability	Requirements should map to deterministic test procedures and observable expected outcomes.

3.3 Constraints and Assumptions

- Raft correctness depends on correct quorum, leader, and log-commit behaviour.
- A local test run does not by itself prove behaviour under production network partitions or hardware

failure.

- TLS tests must distinguish secure defaults from explicitly configured insecure test options.
- LLM explanations can be incomplete or incorrect and must not be treated as authoritative database

state.

- Throughput, latency targets, cluster size limits, and deployment-specific retention policies require

explicit operational values before release.

Laboratory Record: Experiment 1 – Software Requirements Specification (SRS)

Software Requirements Specification (SRS)

> **Aim:** To prepare the Software Requirements Specification for our project kvstore, a distributed, fault-tolerant, transactional key-value store.

Purpose and Scope

kvstore stores string keys with byte values. A client can put, get and delete a key, or group several operations into one atomic transaction. Data is replicated on many nodes using Raft consensus, kept safe with a checksummed write-ahead log, and read through MVCC snapshots. The scope also includes TLS, metrics and a live dashboard, an operational intelligence layer, sharding with two-phase commit across shards, and three command line programs. It is written in Go using only the standard library. Reads that span many shards, trained ML models and fully automatic leader transfer are out of scope.

Stakeholders / User Classes

S. No.	Stakeholder / User	Role
1	Application developers	Store, read and update data through the client library, CLI or HTTP API.
2	Cluster operators	Deploy, monitor and maintain the cluster; add or remove nodes.
3	System integrators	Embed kvstore in larger systems using its protocols and sharding interfaces.
4	Project evaluators and maintainers	Review the SRS, the design and the known limitations.

Assumptions

- A successful fsync really writes the data to disk.
- Clocks of the machines stay within the configured maximum skew of each other.
- The network may delay, drop or reorder messages, but does not alter them unless TLS is used.

Functional Requirements

ID	Functional Requirement	Priority
FR-032	The system shall provide a transactional API with begin, get, put, delete and commit.	High
FR-007	The system shall append every write-ahead-log record with a checksum.	High
FR-015	The system shall mark an operation committed only after a durable majority of nodes, including the leader, has stored it.	High
FR-026	The system shall reject a transaction at commit if a key it writes already has a newer committed version (first-committer-wins).	High
FR-037	The system shall return a not-leader reply with the leader address, when known, if a request reaches a non-leader node.	High
FR-066	The system shall provide a REST-style API for key read/write/delete, transactions, node status and metrics.	High
FR-048	The system shall expose metrics in Prometheus text format and plain JSON.	Medium
FR-073	The system shall provide a dedicated cross-shard endpoint that uses two-phase commit for atomic writes across shards.	Medium

Non-Functional Requirements

ID	Category	Non-Functional Requirement
NFR-001	Performance	A write is committed once a majority of nodes has stored it; it does not wait for every node.
NFR-010	Security	All network traffic can be encrypted with TLS, with optional mutual TLS.
NFR-006	Reliability	A write reported as successful survives a crash while a majority of the cluster remains durable.
NFR-021	Usability	Operators can ask questions about the cluster in plain language through the dashboard or the HTTP API.

Constraints / Business Rules

ID	Constraint / Business Rule
BR-01	Only the Go standard library may be used; no third-party package.
BR-02	Internal packages follow a strict one-way (acyclic) dependency order.
BR-03	An ordinary transaction whose keys are on different shards is rejected; only the dedicated cross-shard endpoint may handle it.
BR-04	The optional LLM may only choose from a fixed set of read-only operations.

External Interfaces

User interface: the kvctl command line client (get, put, delete, status) and a browser dashboard with cluster topology, live charts, key browser, insight feed and a question box.

Inputs: key-value requests, transaction requests, configuration (node addresses, TLS certificates) and natural-language operator questions.

Outputs: values and acknowledgements, transaction outcomes, metrics, and alerts or insights on the dashboard.

Result

In this experiment we prepared the SRS of kvstore with its purpose, scope, user classes, functional and non-functional requirements, constraints and interfaces. Each requirement is written so that it can be tested.

Software Process Model and Development Plan

4.1 Selection Rationale

The Lab 2 artefact evaluates process-model selection for a project with distributed protocols, transaction semantics, recovery requirements, security concerns, and evolving observability features. A purely linear process is less suitable because protocol assumptions and integration risks need repeated validation. A risk-driven, iterative and incremental approach is a good fit: each increment delivers a coherent capability while high-risk behaviours are prototyped and tested early.

4.2 Chosen Approach: Risk-Driven Iterative Incremental Development

The chosen process combines risk-driven planning with iterative and incremental delivery. Each cycle produces a coherent capability, tests high-risk behaviour early, and updates requirements and design records when evidence indicates a change is needed.

1. **Requirements baseline:** identify core API, replication, durability, transaction, security, and observability requirements.
2. **Architecture and risk analysis:** define subsystem boundaries and assess risks such as stale leader routing, WAL recovery, MVCC conflicts, and cross-shard in-doubt outcomes.
3. **Incremental implementation:** build and validate transport/routing, consensus, storage, transactions, and observability in controlled increments.
4. **Integration and regression:** run unit, package, integration, and end-to-end tests at each increment.
5. **Review and refinement:** update requirements, diagrams, interfaces, and test traceability when design decisions change.

4.3 Risk Register

The risk register records the principal failure modes, their likely impact, and the evidence or controls used to reduce them.

Table 4.1: *Risk register for kvstore*

ID	Risk	Impact	Mitigation / evidence
R1	Incorrect leader routing	Writes may be rejected or sent to a follower.	Leader hint, redirect-and-retry tests, and leader-change integration tests.
R2	WAL durability gap	Committed data may be lost after restart.	Durable append before acknowledgement; crash/restart recovery test.
R3	MVCC conflict bug	Lost update or inconsistent transaction result.	Concurrent-write tests and first-committer-wins assertions.
R4	Cross-shard partial commit	Atomicity violation across shards.	Durable decision log, prepare/commit/abort protocol, and crash-recovery tests.
R5	Unsafe operational query	Unexpected or privileged operation.	Restricted command grammar, safe fallback, and negative tests.
R6	Untrusted TLS endpoint	Connection security can be bypassed.	Certificate-chain validation and negative trust test.
R7	LLM provider outage or incorrect answer	Misleading operational advice.	Keep LLM advisory, bound diagnostic context, and handle provider failure without affecting the commit path.

4.4 Case Analyses from the Lab 2 Handout

The process-model exercise compares the fit of five models against requirement stability, early delivery, feedback, risk handling, and testing. The four classroom cases below are retained from the original assignment because they demonstrate why model selection depends on the problem rather than on a universal ranking.

4.4.1 Case 1: College Payroll System – Waterfall

Salary rules, allowances, and deductions are known and major changes are not expected. Requirements are stable and documentable; payroll/HR can review calculations at planned checkpoints. Waterfall is suitable because requirements can be approved up front and calculation rules can be traced through requirements, design, implementation, testing, and deployment. Payroll accuracy still requires thorough validation.

4.4.2 Case 2: New Food-Delivery App – Agile

The startup expects frequent releases and users may request features after trying the application. Ordering, payment, delivery tracking, and restaurant workflows benefit from a minimum viable feature set and continuous feedback. Agile short iterations allow the team to reprioritise a backlog; continuous integration and regression testing protect existing flows.

4.4.3 Case 3: Hospital Emergency Monitoring System – Spiral

The consequences of failure are serious, so technical and safety risks must be studied explicitly. Spiral fits because each cycle defines objectives and alternatives, analyses risks, builds and verifies a solution,

and plans the next cycle. A real medical system would additionally require applicable regulatory, clinical, and safety-engineering processes.

4.4.4 Case 4: New College Portal Interface – Prototyping

Students and staff are uncertain about screens and want samples before finalising requirements. A low-cost interactive mock-up gives stakeholders something concrete to evaluate for navigation, terminology, accessibility, and task completion. The prototype is exploratory unless it is engineered to production quality.

4.5 Decision Factors and Weighted Decision Matrix

The original Lab 2 analysis identifies the following kvstore characteristics: core operations are clear but edge cases, operational intelligence, sharding, and recovery require refinement; technical complexity is high because Raft, MVCC, WAL, HLC, TLS, network transport, and cross-shard coordination interact; failure impact is high because lost or inconsistent data undermines the system; early verification is essential; feedback comes from tests, fault injection, metrics, dashboard behaviour, and review; and the modular Go architecture supports incremental implementation. Ratings are qualitative judgements, not measurements. Each criterion is rated from 1 (weak fit) to 5 (strong fit), then multiplied by its weight.

Table 4.2: *Weighted process-model comparison from the Lab 2 analysis*

Criterion	Weight	Waterfall	Incremental	Prototype	Spiral	Agile
Requirement evolution	15%	4	4	5	4	5
Risk analysis	30%	2	3	2	5	3
Early verification	20%	2	4	4	4	5
Architectural complexity	20%	2	4	2	5	3
Feedback / adaptation	15%	2	3	5	4	5
Weighted score / 5	**100%**	**2.40**	**3.60**	**3.25**	**4.50**	**4.00**

The weighted score is calculated as the sum of each criterion's weight multiplied by its rating, with the weights expressed as proportions. Spiral receives the highest score (4.50/5) because risk analysis has the greatest weight. The original recommendation is therefore Spiral as the primary model, with incremental and Agile-style delivery practices. This is a recommendation based on the project's risk profile, not a claim that the repository historically declared a formal lifecycle.

4.6 Practical Process Controls

The lab defines a repeatable cycle: plan a small testable objective and identify affected SRS requirements; analyse failure modes, impact, likelihood, and mitigation; design interfaces while preserving the modular dependency hierarchy; implement one vertical slice or module increment; verify with unit/integration tests, fault injection, crash-recovery tests, race detection, build and vet checks as appropriate; review test evidence, metrics, defects and limitations; then re-plan according to the evidence. An untested capability must not be treated as verified.

4.7 Development Increments

The increments below group implementation work into deliverables with explicit exit evidence. A capability should be treated as verified only when the corresponding evidence is available.

Table 4.3: *Development increments and exit evidence*

Increment	Deliverable	Exit evidence
Foundation	API, client, and shard routing.	Valid request/response, invalid-request handling, and routing tests.
Replication	Raft log, WAL, and durable commit.	Leader/follower tests, replication acknowledgements, and restart recovery.
Transactions	HLC, MVCC, and transaction API.	Snapshot-read tests and conflict/abort tests.
Distributed transactions	Shard gateway and two-phase commit.	Prepare/commit/abort tests, atomicity checks, and coordinator-recovery tests.
Operations	Metrics, dashboard, and operational query.	Metrics endpoint, safe query fallback, and TLS checks.
AI-assisted diagnostics	Anomaly evidence and LLM hypotheses.	Failure isolation and proof that diagnostics do not affect the database commit path.

Laboratory Record: Experiment 2 – Selecting a Software Process Model

Selecting a Software Process Model

> **Aim:** To compare software process models and select a suitable model for different software projects, including kvstore.

Process Model Comparison

Factor	Waterfall	Incremental	Prototype	Spiral	Agile
Requirements	Stable	Partly stable	Unclear	Can change	Changing
Early delivery	Low	Yes	Sample	Possible	Yes
Customer feedback	Low	Medium	High	High	Very high
Risk handling	Low	Medium	Medium	Very high	Medium

Case Analysis

Case	Model	Reason
1. College Payroll System	Waterfall	Salary rules are already known and will not change much, so requirements can be fixed first and followed step by step.
2. New Food-Delivery App	Agile	Users will ask for new features after trying the app; short releases give early software and regular feedback.
3. Hospital Emergency Monitoring System	Spiral	A failure is serious, so technical and safety risks must be analysed again in every cycle.
4. New College Portal Interface	Prototype	Users are not sure how the screens should look; sample screens give feedback before the requirements are fixed.

Process Model for kvstore

For our project we selected the Spiral model, delivered in small increments with Agile-style feedback. The reasons are:

- Risk is the main concern. Leader changes, crashes, concurrent writes and network faults must be handled correctly, and Spiral looks at risk in every cycle.
- The system can be built and tested in stages: storage and WAL, consensus, MVCC transactions, then security, metrics and sharding.
- The SRS already asks for fault-injection and crash-recovery tests, which fit repeated risk-review cycles better than one single pass.

We also scored the five models with weights (risk analysis 30%, early verification 20%, architectural complexity 20%, requirement evolution 15%, feedback 15%) out of 5:

	Waterfall	Incremental	Prototype	Spiral	Agile
Weighted score	2.40	3.60	3.25	4.50	4.00

Result

We compared the five models and selected Waterfall, Agile, Spiral and Prototype for the four cases. For kvstore, Spiral with incremental delivery scored highest and was selected.

Use-Case Model

5.1 Use-Case Diagram

The use-case diagram is presented in Laboratory Experiment 3 below.

5.2 Actors

- Client Application: performs PUT, GET, DELETE, transaction execution, reads transaction results, and may subscribe to live events.
- Cluster Operator / Administrator: manages membership, TLS/certificates, leadership, fault-injection tests, metrics, AI operational queries, and reports.
- System Administrator: performs broader system configuration and privileged administration.
- External LLM Provider: an external service used by the observability/diagnostic flow; it is not a primary actor in data mutation.

5.3 Principal Use-Case Descriptions

Table 5.1: *Principal use cases and alternative flows*

Use case	Main flow	Alternative / failure flow
Put key–value	Client sends key and value; the system routes to the shard/leader; the entry is replicated and committed; success is returned.	A follower redirects to the leader; validation or quorum failure returns a controlled error.
Get key–value	Client sends a key and the shard returns the visible value.	An absent key returns not-found; a transient routing issue may trigger leader discovery.
Delete key–value	Delete is routed and committed; later reads reflect the deletion semantics.	An absent key follows the API's documented idempotency/not-found policy.
Execute transaction	A begin timestamp is assigned; reads use the snapshot; writes are buffered; commit validates conflicts.	A conflicting transaction is aborted; no partial conflicting write is applied.
Cross-shard transaction	Gateway identifies shards; coordinator durably records the decision; participants prepare; coordinator commits or aborts.	A failed participant causes abort or recovery from the durable coordinator decision.
View metrics dashboard	Operator queries metrics and receives performance/health information.	Unavailable metrics are surfaced as unavailable rather than fabricated.
Ask AI operational query	Supported operational intent is parsed; relevant evidence is assembled; a diagnostic hypothesis is returned.	Unknown/unsafe command falls back to help or an unsupported-command response.
Configure TLS	Administrator supplies certificate configuration; system validates trust.	An untrusted certificate is rejected in secure mode.

5.4 Use-Case Relationships

The supplied diagram uses include relationships for common supporting behaviour such as request authentication and validation, and extension relationships for optional or conditional flows such as leader redirect. The diagram should be read with the system boundary in mind: client operations are

separated from operator-only actions, while shared validation and routing concerns are reused across multiple use cases.

Laboratory Record: Experiment 3 – Use Case Diagram

Use Case Diagram

> **Aim:** To identify the actors and use cases of kvstore and draw the use case diagram.

Actor	Use cases
Client Application	Put Key-Value, Get Key-Value, Delete Key-Value, Execute Transaction, View Transaction Result, Subscribe to Live Events (SSE)
Cluster Operator / Administrator	Add Node, Remove Node, Configure TLS / Certificates, Trigger Leadership Transfer, Run Fault-Injection Test, View Metrics Dashboard, Ask AI Operational Query, Generate Cluster Report
System Administrator	Configure TLS / Certificates, View Metrics Dashboard, Ask AI Operational Query, Generate Cluster Report

Shared use cases are Authenticate Request, Check Leader / Quorum Status and Validate Transaction. Put, Get, Delete, Execute Transaction and Subscribe include Authenticate Request; Put and Execute Transaction include Check Leader / Quorum Status; Execute Transaction includes Validate Transaction. Handle Leader Redirect is the extension of Check Leader / Quorum Status.

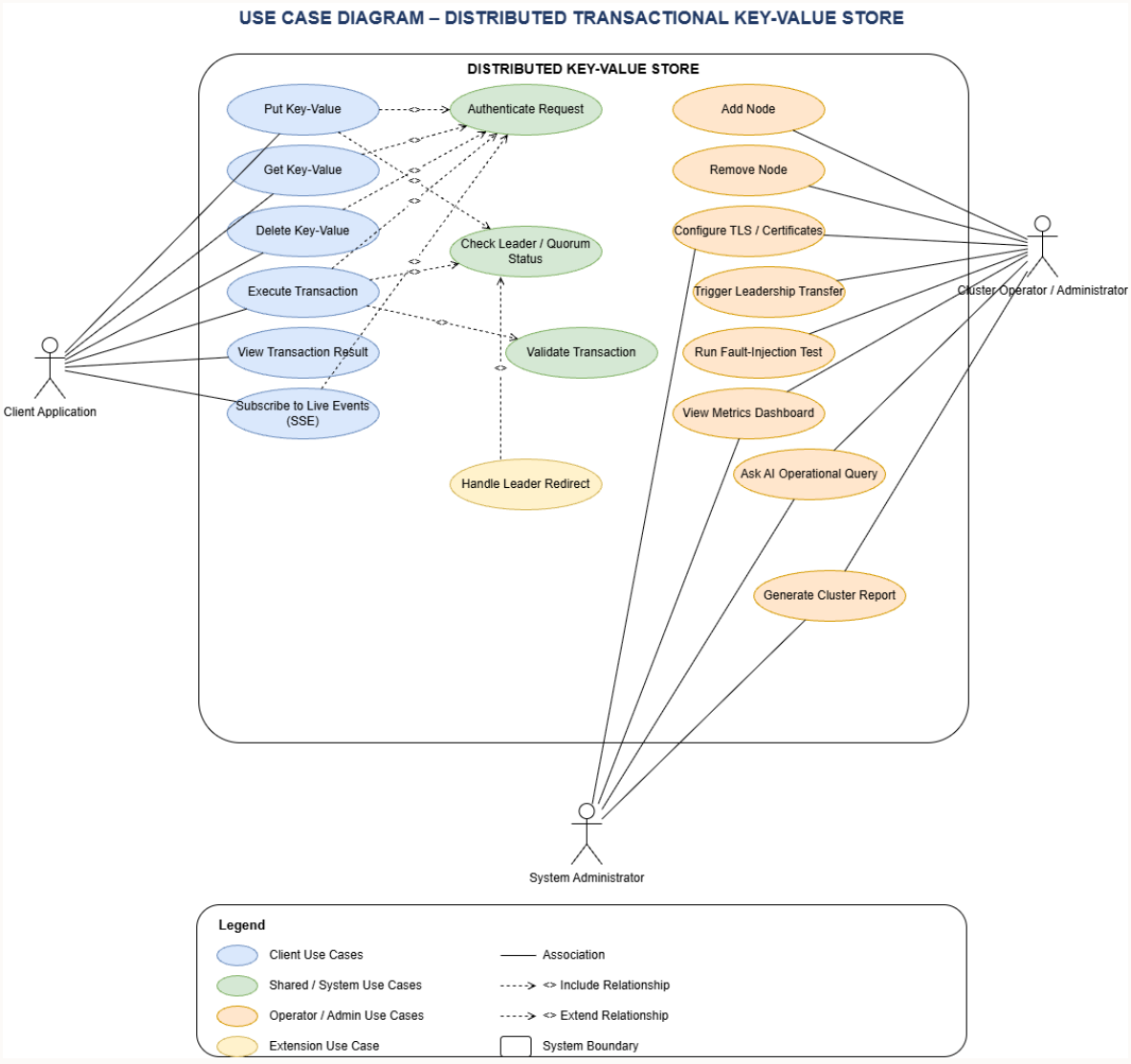


Fig. 3.1 Use case diagram of kvstore

Result

In this experiment we prepared the use case diagram for kvstore. We identified the main actors and use cases and showed their interactions in the diagram.

Data-Flow Modelling

6.1 Level 0: Context Diagram

The Level 0 context diagram is presented in Laboratory Experiment 4 below.

At context level, the complete system is one process, labelled 0.0 kvstore System. The diagram is a boundary view rather than a deployment topology. It names external entities and the information exchanged with each.

Table 6.1: *Level 0 context-diagram entities and flows*

External entity / direction	Flow label	Meaning
Client application → kvstore	KV Requests	Requests for key–value operations and transactions enter the system.
kvstore → client application	Transaction outcomes / data	The result, retrieved data, or transaction outcome returns to the calling client.
Cluster operator ↔ kvstore	Live dashboard	Operational and cluster information is presented to the operator.
kvstore → external LLM provider	Anomaly evidence	Selected operational evidence is sent for diagnostic interpretation.
External LLM provider → kvstore	Root-cause hypotheses	The provider returns hypotheses that may assist an operator; these are advisory and do not control commit decisions.

The system boundary is important: the client and operator interact with kvstore, while the external LLM is a separate provider. The diagram does not grant the LLM authority to mutate data or determine consensus. Its two flows are diagnostic evidence out and hypotheses back. The client-facing data flow remains separate from this optional diagnostic interaction.

6.2 Level 1: Subsystem Data Flow

The Level 1 subsystem DFD is presented in Laboratory Experiment 4 below.

The Level 1 diagram decomposes the single context process into four numbered processes. The incoming and outgoing flows should be read as labelled in the original artefact, rather than inferred from physical placement alone.

Table 6.2: *Level 1 processes, flows, and interpretation*

Process / store	Named flow(s)	Interpretation
External entities	Requests; read outcomes; alerts/insights	Represents the external client/operator boundary. Requests enter API and routing; outcomes return from the system; observability returns alerts and insights to the external side.
1.0 API and Routing	Requests; write proposals	Accepts requests and forwards proposed state changes toward consensus. This is also the boundary where routing and leader redirection are handled.
2.0 Consensus	Write proposals; committed entries; fsync append	Receives proposed operations, coordinates Raft replication and commitment, and relies on durable log append before treating records as safely persisted.
3.0 Storage	Committed entries; metrics streams; read outcomes	Applies committed entries to state, maintains key/value and version state, serves read outcomes, and contributes performance/operation signals.
4.0 Observability	Metrics streams; alerts/insights	Consumes measurements and produces operator-facing insights and alerts. The AI/diagnostic path is an adjunct to operations, not the commit authority.
D1: WAL / durable log	Fsync append	Durable data is written by the log path before the system acknowledges the corresponding durable commit point.

6.3 Level 2: Internal Package Architecture

The Level 2 package-architecture DFD is presented in Laboratory Experiment 4 below.

The Level 2 view expands the major processes into internal package groups. The table describes the responsibility and principal flow represented in the supplied artefact.

Table 6.3: *Level 2 package groups and responsibilities*

Package group	Components shown	Responsibility and flow
API and transport	HTTP API, client protocol, wire envelope	Accepts and validates requests, serializes replies, and communicates leader hints and errors.
Routing	Client library; shard router/ring	Routes requests to the correct shard; the client library handles redirects and retry hints.
Consensus and durability	Raft; WAL	Coordinates leader election, replication, commit advancement, durable append, and recovery replay.
Storage and transactions	Storage engine; HLC; MVCC	Applies committed commands, assigns ordered timestamps, serves snapshot reads, and validates write conflicts.
Distributed transactions	Shard gateway; two-phase commit coordinator; decision log	Coordinates participants and recovers the durable commit/abort decision.
Observability	Metrics registry; operational intelligence	Collects performance signals, exposes metrics, and produces bounded advisory diagnostics.
Security configuration	TLS/certificate configuration	Establishes trusted transport and rejects invalid certificate chains.

6.4 Level 2 Internal Package Interpretation

The Level 2 diagram divides the system into package groups. The package labels below preserve the names shown in the supplied diagram, including implementation-level labels where present.

Table 6.4: *Level 2 package interpretation and internal flows*

Process / package group	Components shown	Responsibility and flow
1.0 API and Routing	1.1 `httpapi/transport`; 1.2 client; 1.3 shard	Transport accepts and encodes requests; the client library handles calls and leader hints; shard routing maps keys to the appropriate cluster/shard. The diagram shows client-to-Raft leader redirect and transport/shard interactions.
2.0 Consensus	2.1 `raft`; 2.2 `wal` (Write-Ahead Log)	Raft handles leader election, proposals, replication, and commitment. The WAL stores records for durable recovery. Proposals flow to the WAL; committed entries flow from Raft toward storage.
3.0 Storage	3.1 `storage`; 3.2 `hlc` (Clock); 3.3 `mvcc` (Multi-Version Concurrency Control)	Storage applies committed operations; HLC supplies timestamp values; MVCC manages version chains, snapshot visibility, and conflict checks.
4.0 Observability	4.1 `metrics`; 4.2 `ai`	Metrics collects performance measurements and exposes a statistical interface. AI/operational intelligence consumes allowed statistics and produces insight feeds.

6.4.1 Named internal flows

- Leader Redirect: a request received by a non-leader can be answered with a leader hint so the client can retry against the current leader.
- Proposals: the client/transport path submits candidate operations to the Raft subsystem for ordered replication.
- Committed Entries: after the consensus commit condition is met, entries are applied by storage.
- Fsync Append: the write-ahead log persists records before the system relies on them for crash recovery.
- Performance Measures: storage/cluster activity contributes counters and latency measurements to the metrics component.
- Statistical Interface: metrics are made available to the operational intelligence component through a narrow interface rather than direct access to database internals.
- Insight Feeds: operational findings are surfaced to the API/dashboard path for the operator.
- httpapi/transport: protocol boundary and request/response handling.
- client: client-side calls, leader hint caching, and retry/redirect logic.

- shard: key-to-shard routing and shard-level dispatch.
- raft: leader election, log replication, and commit progression.
- wal: append-only durable records and replay support.
- storage: application of committed entries and durable state management.
- hlc: hybrid logical timestamps used by version and transaction logic.
- mvcc: version chains, snapshot reads, write-set validation, and conflict checks.
- metrics: counters, latency measurements, histograms, and metric exposition.
- ai: anomaly interpretation and bounded operational query assistance.

6.5 Balancing and Consistency Check

A balanced DFD preserves the external flows when moving from context to Level 1: client requests still enter the system and results still return to the client; operator-facing insights still originate from observability; and diagnostic evidence/hypotheses remain an external interaction. Level 2 decomposes internal subsystems without inventing a new external actor. Data stores are accessed through processes, not directly by external entities.

Laboratory Record: Experiment 4 – Data Flow Diagram (DFD)

Data Flow Diagram (DFD)

> **Aim:** To prepare the Context Diagram, the Level 1 data flow diagram and the Level 2 internal package diagram for kvstore.

External Entities

S. No.	External Entity	Data sent to system	Data received from system
1	Client Application	Put, Get, Delete and transaction requests	Values, transaction outcomes, errors
2	Cluster Operator	Monitoring queries	Live dashboard, metrics, alerts, insights
3	External LLM Provider	Root-cause hypotheses	Anomaly evidence

Processes and Data Store

No.	Process	What it does
1.0	API & Routing	Takes client requests, checks them, follows leader redirects and finds the shard.
2.0	Consensus	Raft leader election and log replication; commits proposals using the WAL.
3.0	Storage	Applies committed entries and keeps key-value data with HLC timestamps and MVCC.
4.0	Observability	Collects metrics and raises alerts and AI-assisted insights.
D1	Persistent Storage	Committed key-value entries, WAL records and MVCC versions.

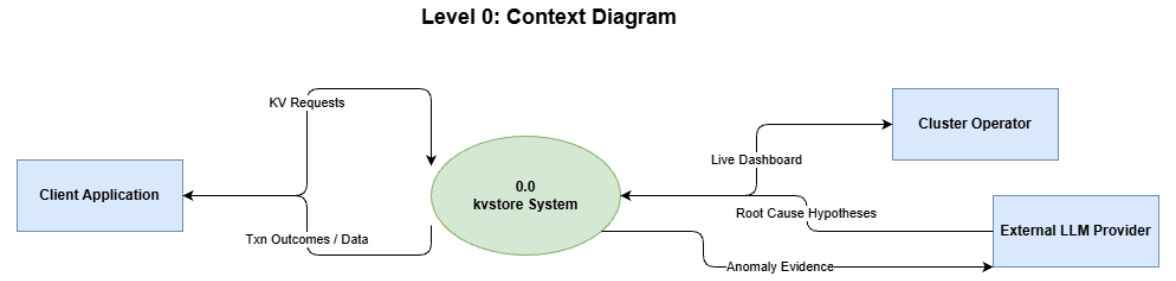


Fig. 4.1 Level 0: Context diagram

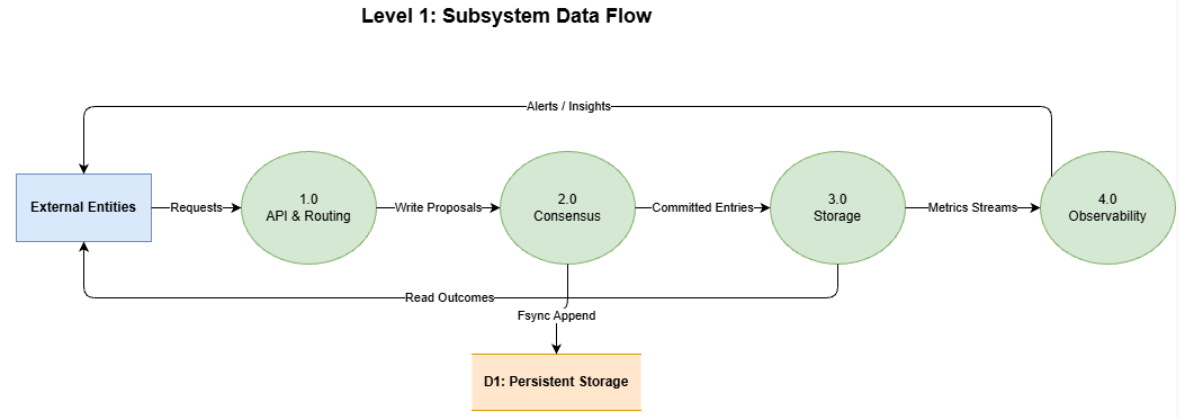


Fig. 4.2 Level 1: Subsystem data flow

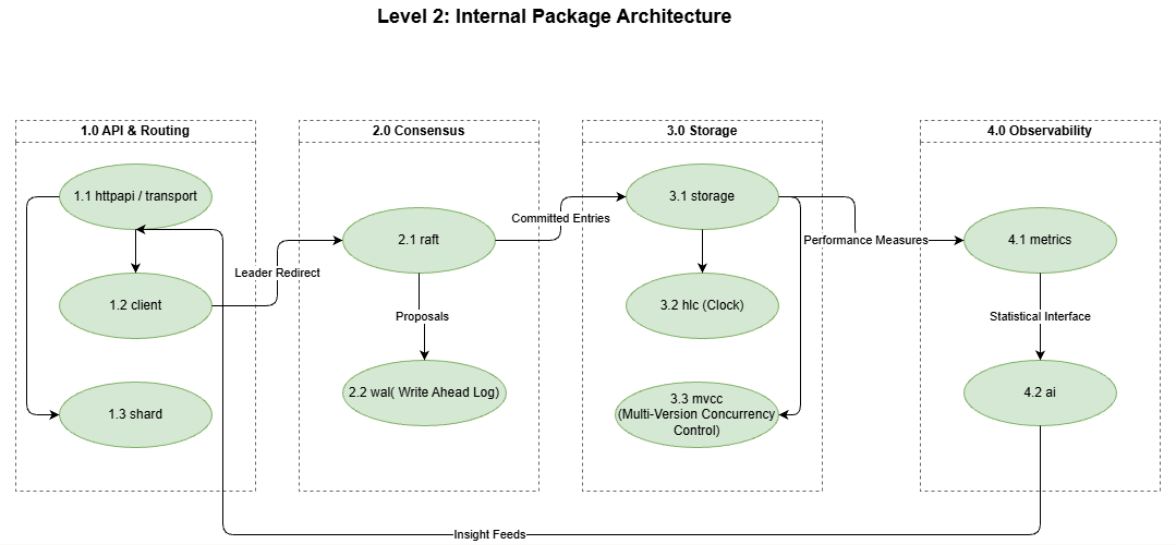


Fig. 4.3 Level 2: Internal package architecture

The four Level 1 processes are the same four groups shown in Level 2, and the flows match across the levels, so the diagrams are balanced.

Result

In this experiment we prepared the Context Diagram, the Level 1 DFD and the Level 2 package diagram for kvstore, showing the external entities, processes, data flows and the data store.

UML Structural Design

7.1 Class Diagram

The UML class diagram is presented in Laboratory Experiment 5 below.

7.2 Important Classes and Responsibilities

Table 7.1: *Important classes and their responsibilities*

Class / component	Responsibility	Collaborators / key data
Engine / transaction engine	Exposes Begin, Get, Put, Delete, Rollback, and Commit operations; coordinates transactional work.	MVCC version store, HLC, consensus, context and transaction handle.
Txn	Represents one transaction, its snapshot, buffered writes, and final outcome.	Read timestamp, write set, commit/abort result.
MVCC version store	Maintains version chains, snapshot visibility, and write-conflict checks.	Keys, values, timestamps, transaction read/write sets.
Hybrid logical clock (HLC)	Produces and compares ordered timestamps for transaction visibility and ordering.	Physical time component and logical counter.
Raft / consensus node	Elects leaders, replicates log entries, and advances the commit index.	Proposals, log entries, acknowledgements, leader state.
Write-ahead log (WAL)	Appends durable records, checksums entries, and replays records during recovery.	Log records, fsync/append result, replay stream.
Shard ring / router	Maps keys deterministically to a shard.	Key, shard set, shard identifier.
Two-phase commit coordinator	Coordinates prepare and durable commit/abort across shard participants.	Participant list, prepare votes, durable decision record.
StatsProvider / metrics adapter	Exposes operational measurements without exposing storage internals.	Counters, gauges, histograms, node observations, version count.

7.3 Design Principles Reflected

- Separation of concerns: transport, consensus, persistence, transactions, and metrics have

different responsibilities.

- Dependency direction: the API layer delegates to internal services rather than implementing consensus or storage logic itself.
- Encapsulation: the WAL and MVCC store own their data structures and expose operations rather than exposing raw internals.
- Explicit transaction state: transaction and cross-shard transaction objects make outcome and recovery responsibilities visible.
- Operational isolation: metrics and AI diagnostics observe system behaviour without becoming commit authorities.

Laboratory Record: Experiment 5 – UML Class Diagram

UML Class Diagram

> **Aim:** To identify the major classes of kvstore and prepare a UML class diagram with attributes, methods, relationships and multiplicities.

Classes, Attributes and Methods

Class	Main attributes	Main methods
RaftCluster	members: ClusterNode[], leaderID	electLeader()
ClusterNode	role, currentTerm	startElection(), becomeLeader()
HybridLogicalClock	physicalTime	now()
RaftLog	entries: LogEntry[]	appendEntry(e)
LogEntry	term, checksum	verifyChecksum()
WriteAheadLog	filePath	appendRecord(data), flush()
MVCCVersionStore	versionChains	readVersion(key, ts), writeVersion(key, value, ts)
KeyValueVersion	timestamp, deleted	isVisible(readTs)
Transaction	beginTimestamp, status	begin(), commit()
CrossShardTransaction	decisionRecorded	prepareAllShards()
MetricsCollector	histograms	recordLatency(op, ms)
OperationalIntelligence Engine	anomalyThreshold	detectAnomaly(metric), forecastTrend(series)
TLSConfig	mutualTLSEnabled	loadCertificate()
ClientLibrary	currentLeaderAddress	get(key), put(key, value)
HttpApiLayer	routes	handleRequest(req)
ShardGateway	shardList: RaftCluster[]	routeKey(key)
TwoPhaseCommit Coordinator	decisionLog	prepare(txn), commitOrAbort(txn)
StatsProvider *(interface)*	–	getStats()
Executor *(interface)*	–	execute(operation)

Relationships

RaftCluster has 1..* ClusterNode (composition). Each ClusterNode owns one HybridLogicalClock, RaftLog, WriteAheadLog, MVCCVersionStore and MetricsCollector (composition); RaftLog has 0..* LogEntry and MVCCVersionStore has 0..* KeyValueVersion. ClusterNode has 0..1 OperationalIntelligenceEngine, TLSConfig and HttpApiLayer (aggregation). CrossShardTransaction inherits from Transaction. ShardGateway uses 1..* RaftCluster and owns one TwoPhaseCommitCoordinator. HttpApiLayer implements StatsProvider and Executor, and the OperationalIntelligenceEngine depends on both.

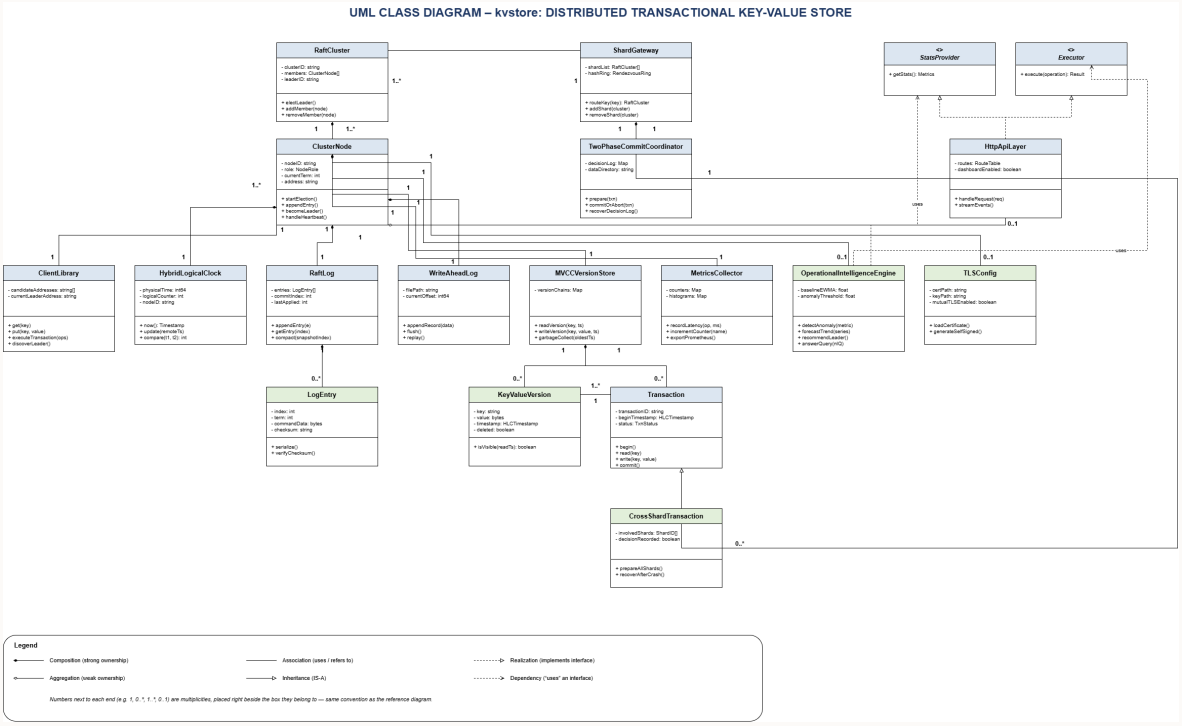


Fig. 5.1 UML class diagram of kvstore

Result: In this experiment we identified the major classes of kvstore and prepared a UML class diagram showing their attributes, methods, relationships and multiplicities.

Interaction and Sequence Design

8.1 Scenario 1: PUT with Leader Redirect

The corresponding sequence diagram is presented in Laboratory Experiment 6 below.

Narrative: The client library sends a PUT request to the contacted node. If that node is the leader, it appends the log entry, ensures durable persistence, applies the version, and returns success. If it is a follower, it returns a leader hint; the client caches the hint and retries against the leader. This makes redirect-and-retry behaviour explicit rather than assuming every client initially contacts the leader.

8.2 Scenario 2: Transaction with Snapshot Isolation

The corresponding sequence diagram is presented in Laboratory Experiment 6 below.

Narrative: A transaction begins and receives a begin timestamp. Reads use the snapshot visible at that timestamp; writes are buffered. At commit, the system performs a pre-check and proposes the write set through the replicated path. If no conflicting committed write invalidates the transaction, the transaction commits. Otherwise, the transaction is aborted and its intents are released. A conflict must not partially overwrite the winning transaction's state.

8.3 Scenario 3: Cross-Shard Two-Phase Commit

The corresponding sequence diagram is presented in Laboratory Experiment 6 below.

Narrative: The gateway determines which shards participate and sends the transaction to the coordinator. The coordinator durably records the transaction's involved shards before contacting participants. Each shard prepares and holds the relevant intent locks. If all participants prepare, the coordinator records a durable commit decision and asks all participants to commit. If any participant declines, the coordinator records abort and releases participant intents. On restart, the coordinator consults the durable decision log to resolve an in-doubt transaction.

8.4 Cross-Scenario Invariants

The scenarios share the following correctness invariants:

1. A client receives success only after the system reaches the required commit point.
2. A follower redirect does not itself mutate committed state.
3. Snapshot reads are governed by the transaction's visibility timestamp.
4. A conflicting transaction must abort rather than overwrite a committed conflicting write.
5. A cross-shard transaction must not report a successful final commit if only a subset of participating shards committed.
6. A durable decision record is the recovery authority for an in-doubt cross-shard transaction.

Laboratory Record: Experiment 6 – Sequence Diagrams

Sequence Diagrams

> **Aim:** To prepare sequence diagrams for three important scenarios of kvstore and show the order of messages between the user and the system objects.

No.	Scenario	Actor	Lifelines / objects	Main messages

1	PUT with leader redirect	Client application	ClientLibrary; contacted node; leader node; WAL; MVCC store	`put()`
				`appendRecord()`
				`writeVersion()`
				`notLeader(hint)`
				retry at leader

2	Snapshot-isolated transaction	Client application	ClientLibrary; leader node; MVCC store; Raft cluster; follower node	`begin()`
				`get()/put()`
				`commit()`
				`optimisticPreCheck()`
				`proposeWriteSet()`
				`replicateLogEntry()`
				`deterministicApply()`

3	Cross-shard two-phase commit	Client application	Shard gateway; 2PC coordinator; decision-log WAL; Shard A; Shard B	`routeKey()`
				`coordinate()`
				`recordInvolvedShards()`
				`prepare()`
				`recordCommitDecision()`
				`commit()/abort()`

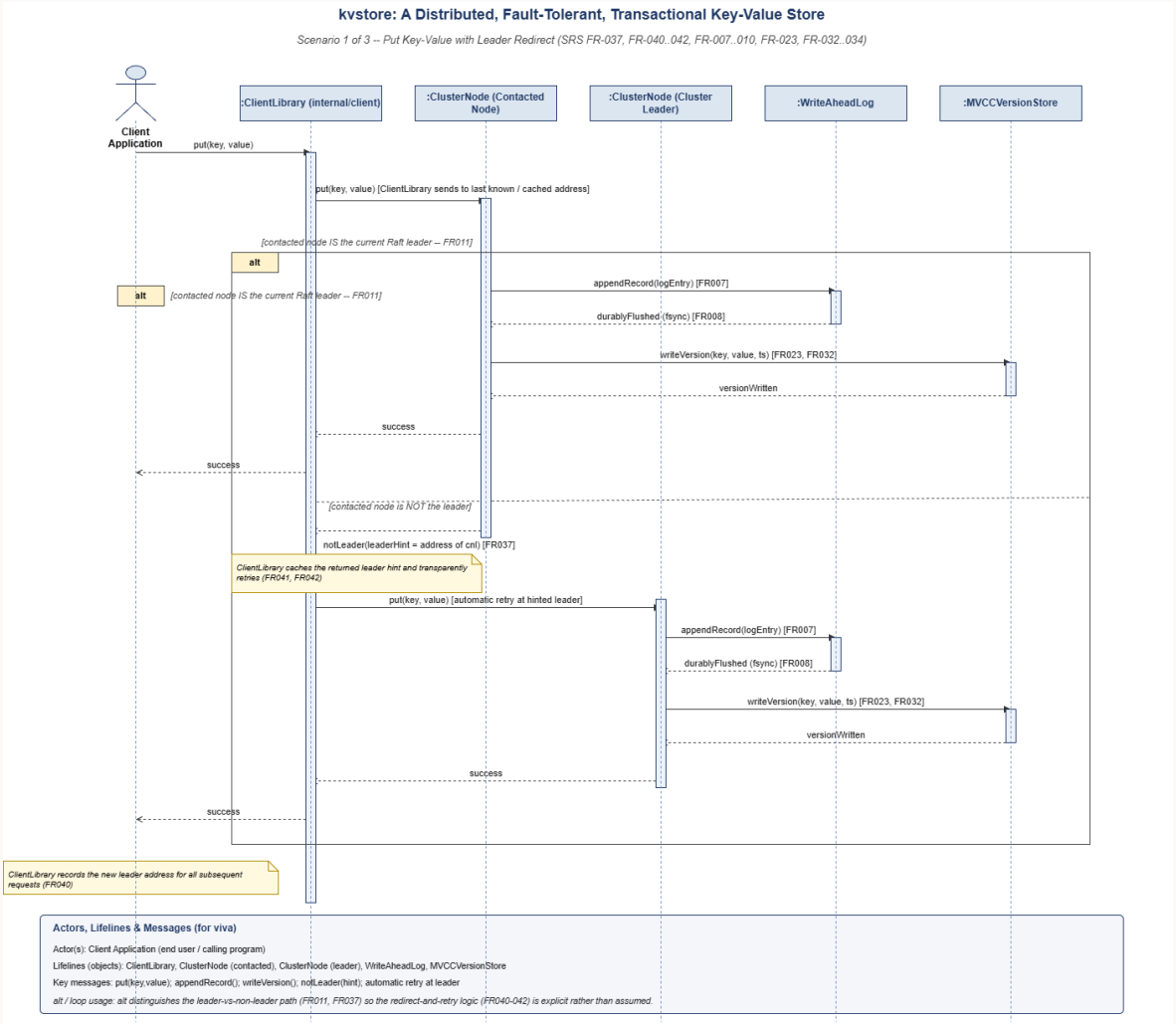


Fig. 6.1 Scenario 1: Put key-value with leader redirect. The alt fragment separates the case where the contacted node is the leader from the case where it returns a leader hint and the client retries.

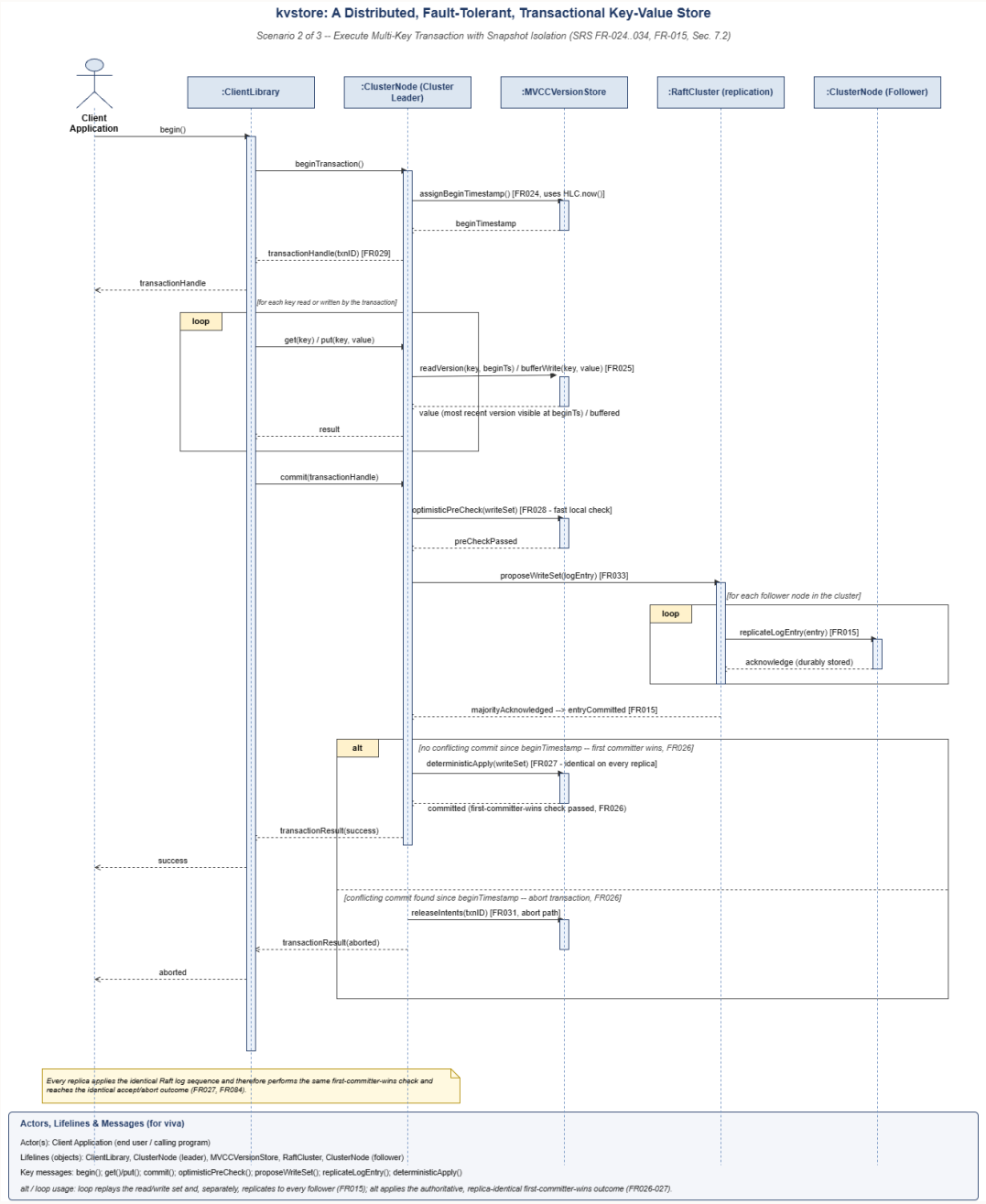


Fig. 6.2 Scenario 2: Multi-key transaction with snapshot isolation. The loop covers reads and writes and replication to followers; the alt fragment shows commit when no conflict exists and abort when another transaction committed first.

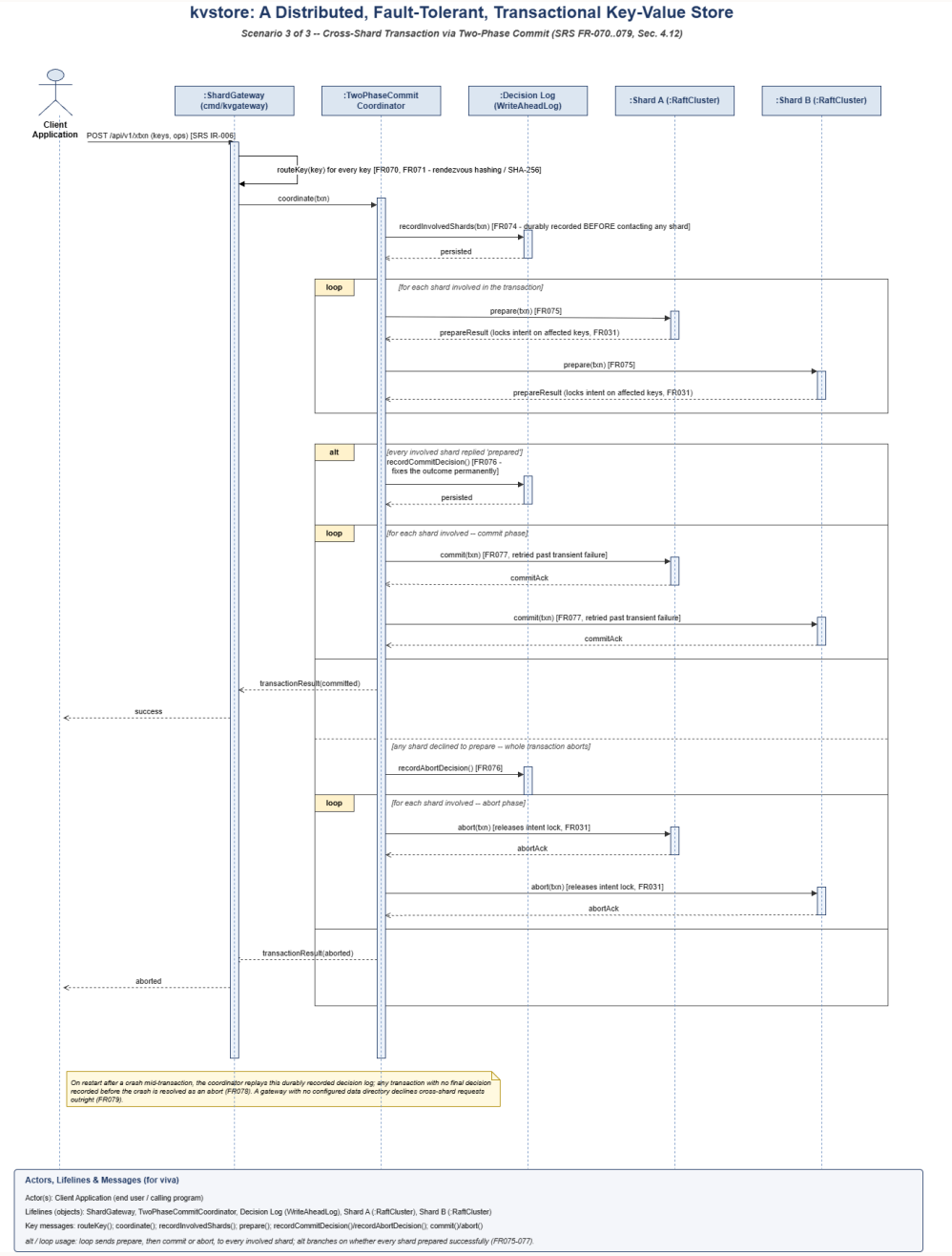


Fig. 6.3 Scenario 3: Cross-shard transaction using two-phase commit. The coordinator records the decision durably before telling the shards to commit or abort.

Result: In this experiment we prepared three sequence diagrams for kvstore and showed the actors, lifelines and the order of messages in each scenario.

Modular Design

9.1 Design Objectives

The modular design exercise focuses on high cohesion and low coupling. Each module should have one main reason to change, expose a small interface, and avoid depending on implementation details owned by another module.

9.2 Module Architecture

The module architecture diagram is presented in Laboratory Experiment 8 below.

9.3 Module Responsibilities and Interfaces

Table 9.1: *Module responsibilities and interface contracts*

Module	Responsibility	Interface / data exchanged
HTTP API / Transport	Accepts requests, validates input, and serializes responses.	Request objects, status codes, response payloads.
Client Library	Provides convenient client operations and redirect handling.	PUT/GET/DELETE, transaction handles, leader hints.
Shard Router	Maps keys to shard ownership.	Key, shard ID, route result.
Raft / Consensus	Performs leader election and replicated-log commitment.	Proposals, log entries, acknowledgements, commit index.
WAL	Provides durable append and replay.	Log record, append result, replay stream.
Storage operations	Applies committed state changes.	Committed command and key/value state.
HLC	Produces ordered hybrid timestamps.	Timestamp generation and comparison.
MVCC	Implements version visibility and write-conflict checks.	Read timestamp, key/version, write set, commit/abort result.
2PC Coordinator	Coordinates distributed transaction outcomes.	Participant list, prepare response, durable decision.
Metrics	Collects and exports operational measurements.	Counters, histograms, scrape endpoint.
Operational Intelligence	Parses bounded operational queries and forms diagnostic context.	Query intent, metric evidence, hypothesis result.
TLS Configuration	Loads and validates transport trust material.	Certificate configuration and secure transport settings.

9.4 Cohesion and Coupling Review

- High cohesion: WAL operations remain in the WAL module; version visibility and conflict rules remain in MVCC; metrics collection remains in the metrics module.

- Low coupling: modules communicate through request types and method contracts rather than reaching into one another's private state.
- Replaceability: the external LLM provider can be changed or unavailable without changing Raft commitment or storage correctness.
- Testability: modules can be unit-tested with controlled dependencies, while integration tests verify the contracts between modules.
- Change isolation: transport changes should not require changes to transaction conflict logic; metrics presentation changes should not modify the WAL format.

9.5 Architecture Decisions

Table 9.2: *Architecture decisions and trade-offs*

Decision	Reason	Trade-off
Raft for replicated log	Provides a clear leader-based replicated-log model.	Requires quorum for progress and careful leader-change handling.
WAL for durability	Enables recovery by replaying durable records.	Adds write-path I/O and log lifecycle work.
MVCC for transaction reads	Supports snapshot reads and conflict checks.	Requires version management and cleanup strategy.
Two-phase commit across shards	Coordinates atomic outcome across shard participants.	Can block while a durable decision is unresolved.
Separate observability path	Protects the correctness path from diagnostic dependencies.	Requires careful context assembly and failure handling.
LLM as advisory assistant	Can provide operator-facing hypotheses from metrics/evidence.	Outputs may be wrong and must not be executed as authority.

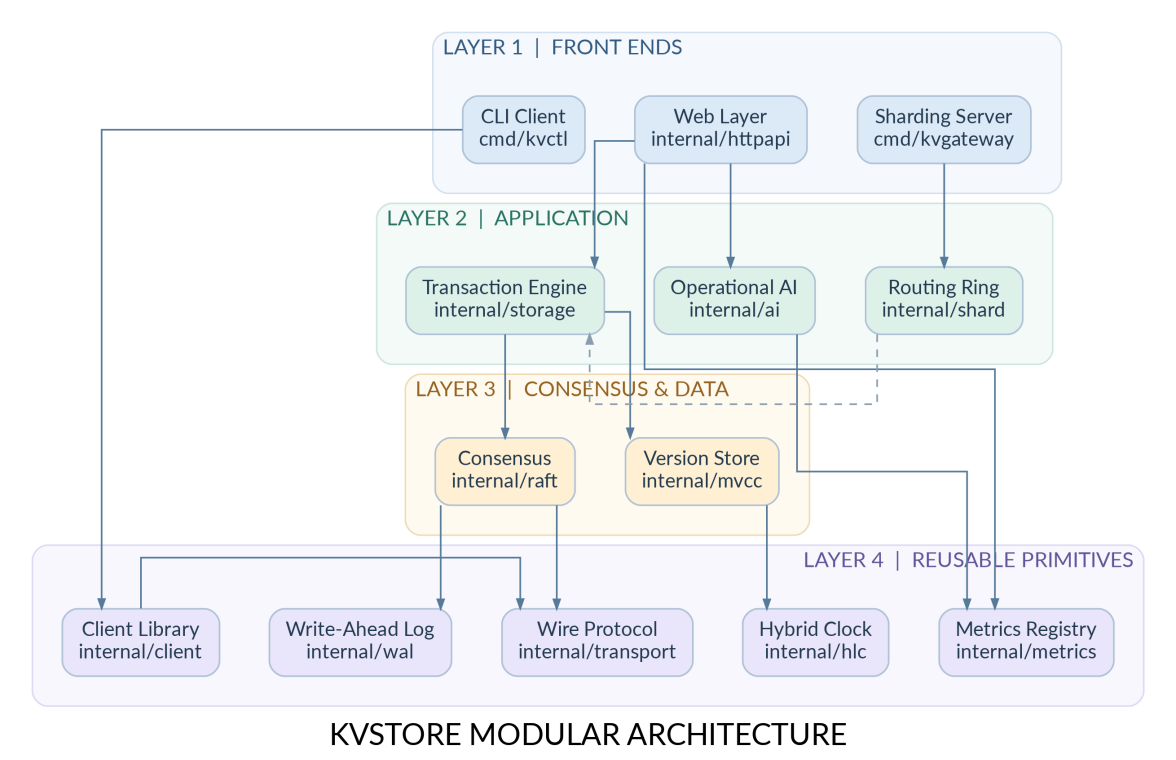
Laboratory Record: Experiment 8 – Modular Design of a Software System

Modular Design of a Software System

> **Aim:** To divide kvstore into modules, define interfaces and justify cohesion and coupling.

Modules

Module	Package	Main responsibility
CLI Client	cmd/kvctl	Command-line read, write, delete and status
Web Layer	internal/httpapi	REST API, live events, dashboard and wiring of modules
Sharding Server	cmd/kvgateway	One gateway address in front of the shard clusters
Transaction Engine	internal/storage	Begin, get, put, delete, commit and rollback
Operational AI	internal/ai	Anomaly detection, forecasting and advice from statistics
Routing Ring	internal/shard	Maps each key to a shard (rendezvous hashing)
Consensus	internal/raft	Raft elections, log replication and commit
Version Store	internal/mvcc	Version chains, snapshot reads and conflict checks
Hybrid Clock	internal/hlc	Hybrid logical clock timestamps
WriteAheadL	internal/wal	Checksummed durable log and replay after restart
Wire Protocol	internal/transport	Request and reply envelopes between clients and nodes
Client Library	internal/client	Leader discovery, redirect and retry
Metrics Registry	internal/metrics	Counters, gauges, histograms and export



KVSTORE MODULAR ARCHITECTURE

Fig. 8.1 Modular design diagram of kvstore (an arrow means "depends on")

Interfaces

Calling → Providing module	Interface	Input	Output
Web Layer → Transaction Engine	Begin, Get, Put, Delete, Commit, Rollback	key, value, context	value, or commit timestamp / error
Client Library, Consensus → Wire Protocol	Envelope / EnvelopeReply	request kind and arguments	reply, NotLeader flag with leader hint
Web Layer adapter → Operational AI	StatsProvider	–	metrics snapshot, node observations, version count
Sharding Server → Routing Ring	ShardFor(key)	key string	shard ID

Cohesion and coupling

Every module has one job: the WAL only stores opaque records and Operational AI only analyses statistics. Coupling is kept low because Operational AI uses the small StatsProvider interface and does not import Raft or storage, and Consensus handles opaque operations. The graph has 14 connections among 13 modules, so average coupling is 14/13 = 1.08. Instability I = Fout / (Fin + Fout) is 0.33 for Wire Protocol, 0.75 for Transaction Engine and 0 for Hybrid Clock and WAL.

Result

In this experiment we divided kvstore into 13 production modules, defined four interfaces and checked cohesion and coupling.

Detailed Modular Design Record

Expanded Lab 8 Modular-Design Record

This appendix retains the important project-specific detail from the original Lab 8 modular-design submission instead of reducing it to a one-page summary.

9.6 Project Requirements Used for Modularisation

Table 9.3: *Requirements informing modular boundaries*

ID	Requirement	Meaning for module design
R1	Key–value operations	Put, Get, Delete, and atomic multi-key transactions.
R2	Fault-tolerant replication	Raft replication across machines; the cluster should continue to make progress when a quorum is available.
R3	Strong isolation	Snapshot isolation and first-committer-wins conflict detection.
R4	Durability	Accepted changes are written to a checksummed durable log before the relevant acknowledgement.
R5	Deterministic ordering	HLC timestamps provide a total order that nodes can use consistently.
R6	Observability	Prometheus/JSON metrics, REST API, and a live dashboard.
R7	Operational intelligence	Statistical anomaly detection, correlation, and bounded diagnostic queries.
R8	Horizontal scaling	Independent shard clusters behind a gateway, with cross-shard coordination where configured.

9.7 Responsibility Inventory

Table 9.4: *Module responsibility inventory*

Module	Responsibility
CLI Client	Command-line operations and client-facing status.
Client Library	Request/reply envelope, leader discovery, redirect, and retry.
Web Layer	REST API, server-sent events, dashboard, and module wiring.
Sharding Server	Gateway routing to shard clusters.
Transaction Engine	Transaction lifecycle and coordination with storage/consensus.
Operational AI	Bounded operational query handling and advisory diagnostics.
Routing Ring	Deterministic key-to-shard assignment.
Consensus	Elections, replication, commit, and snapshot/leadership operations.
Version Store	Version chains, snapshot visibility, and conflict validation.
Hybrid Clock	Timestamp generation and ordering.
WriteAheadLog	Durable append, checksums, fsync, and replay.
Wire Protocol	Request/reply envelopes and leader hints.
Metrics Registry	Counters, gauges, histograms, and exposition.
Fault Injection	Test-only partitions, process kills, and delays.

9.8 Thirteen Modules and Package Mapping

Table 9.5: *Thirteen modules and package mapping*

Module	Package	Primary responsibility
CLI Client	`cmd/kvctl`	Command-line interface for reading, writing, and transaction requests.
Client Library	`internal/client`	Client operations, request/reply handling, and leader redirect retry.
Web Layer	`internal/httpapi`	REST API, Server-Sent Events, and dashboard adapter.
Sharding Server	`cmd/kvgateway`	Single logical gateway in front of shard clusters.
Transaction Engine	`internal/storage`	Transaction API, orchestration, and coordination with persistence/consensus.
Operational AI	`internal/ai`	Metric analysis, anomaly detection, and bounded diagnostic query handling.
Routing Ring	`internal/shard`	Deterministic key-to-shard routing using the configured scoring scheme.
Consensus	`internal/raft`	Elections, replication, snapshots, and leadership transfer.
Version Store	`internal/mvcc`	Version chains, snapshot isolation, and write-conflict validation.
Hybrid Clock	`internal/hlc`	Hybrid logical timestamps and ordering.
WriteAheadLog	`internal/wal`	Checksummed append, fsync, and replay.
Wire Protocol	`internal/transport`	Envelope encoding, request/reply transport, and leader hints.
Metrics Registry	`internal/metrics`	Counters, gauges, histograms, and metric exposition.
Fault Injection	`internal/fault`	Test-only simulated partitions, process kills, and delays.

The package map contains the thirteen production modules named in the Lab 8 artefact. Fault injection is a test-only support capability and is excluded from the production module count and coupling calculations.

9.9 Module Interfaces

9.9.1 Interface 1: Transaction API

The storage engine exposes a transaction handle and operations. The interface shown in the original lab is normalised below for readability; names and responsibilities are retained.

```
Engine.Begin(ctx) returns (*Txn, error)
Txn.Get(key) returns (value []byte, ok bool)
Txn.Put(key, value)
Txn.Delete(key)
Txn.Rollback()
Txn.Commit(ctx) returns (hlc.Timestamp, error)
```

Table 9.6: *Transaction API calls and outputs*

Call	Input	Output / participating modules
Begin	Context	Transaction handle with snapshot timestamp; Web Layer → Transaction Engine.
Get	Key	Value bytes or not-found; Web Layer → Transaction Engine.
Put/Delete	Key and optional value	Change buffered in transaction; Web Layer → Transaction Engine.
Commit	Open transaction	Commit timestamp or conflict abort; Transaction Engine → Consensus and Version Store.
Rollback	Open transaction	Transaction changes discarded; Web Layer → Transaction Engine.

9.9.2 Interface 2: Wire Envelope

The transport envelope distinguishes Raft traffic (RequestVote, AppendEntries, InstallSnapshot, TimeoutNow) from client requests (Get, Put, Delete, transaction). The reply carries an error string and can include NotLeader=true plus LeaderHint so a client can retry. One request uses a fresh TCP connection and receives one envelope reply before the connection closes, according to the lab interface description.

9.9.3 Interface 3: StatsProvider

The operational AI is deliberately limited to a small statistics interface implemented by an adapter in the HTTP layer: type S t a t s P r o v i d e r i n t e r f a c e {

M e t r i c s S n a p s h o t ( ) m e t r i c s . Snapshot

NodeObservations ( ) [ ] NodeObservation

VersionCount ( ) int }

Metrics snapshots provide counters, gauges, and histograms; node observations provide latency, errors, replication lag, and queue depth; version count provides a signal for MVCC garbage-collection pressure. The interface prevents the AI package from directly depending on Raft or storage internals.

9.9.4 Interface 4: Key Routing

```
NewRing(shards []ID) *Ring
(r *Ring) ShardFor(key string) ID
```

The routing contract is deterministic: the same key and same shard set return the same shard on each machine and after restart. The lab specifies that cross-shard requests must not be silently applied non-atomically; they are either handled through the dedicated coordinator path or rejected clearly where that path is not configured.

9.10 Cohesion and Coupling Decisions

The following decisions keep each module cohesive and limit dependencies across architectural boundaries.

Table 9.7: *Cohesion and coupling decisions*

Decision	Rationale	Boundary preserved
C1 – WAL as a generic append-only log	WAL appends, checksums, fsyncs, and replays arbitrary byte records; it does not parse key/value semantics, transaction meaning, or Raft commands. This avoids unrelated reasons to change and enables independent testing.	Log mechanics remain separate from domain semantics.
C2 – Operational AI as a statistics consumer	AI contains analysis/query logic but does not own network transport, persistence, or storage operations.	Narrow statistics interface between diagnostics and the core database.
D1 – Keep HLC independent of storage	The clock returns a timestamp type and does not import MVCC or storage.	Stable ordering primitive remains independently testable.
D2 – Keep shard routing deterministic and stateless	The gateway asks the routing ring for the shard responsible for a key. It does not duplicate transaction logic or depend on storage internals.	Deterministic key-to-shard mapping remains isolated.

9.11 Coupling Numericals from Lab 8

The original module graph counts 14 directed production dependencies across 13 drawn modules. Average coupling is calculated as the number of directed module dependencies divided by the number of modules in the graph:

$$C_{\text{avg}} = \frac{E}{N} = \frac{14}{13} \approx 1.08 \tag{9.1}$$

This is an average graph connection count, not a universal quality score. The graph is sparse and layered; the Fault-Injection module is test-only and its arrow is omitted from the production diagram.

The lab defines module instability as the ratio of outgoing dependencies to the sum of incoming and outgoing dependencies:

$$I = \frac{F_{\text{out}}}{F_{\text{in}} + F_{\text{out}}} \tag{9.2}$$

where F_{in} is the number of modules depending on the module and F_{out} is the number of modules it depends on. $I = 0$ represents a stable primitive and $I = 1$ a volatile top layer.

Table 9.8: *Module instability calculations from Lab 8*

Module	F_{in}	F_{out}	I	Interpretation recorded in Lab 8
CLI Client	0	1	1.00	Top of stack; most volatile.
Sharding Server	0	2	1.00	Top-level gateway.
Web Layer	0	3	1.00	Composition root by design.
Transaction Engine	1	3	0.75	High-level orchestrator.
Consensus	1	2	0.67	Middle layer.
Version Store	1	1	0.50	Balanced.
Client Library	2	1	0.33	Fairly stable.
Wire Protocol	2	1	0.33	Fairly stable.
Hybrid Clock	2	0	0.00	Stable primitive.
Metrics Registry	2	0	0.00	Stable primitive.
WriteAheadLog	1	0	0.00	Stable primitive.
Operational AI	1	0	0.00	Stable server-side leaf behind an interface.
Routing Ring	1	0	0.00	Stable server-side leaf.

For example, Wire Protocol has $F_{in} = 2$ and $F_{out} = 1$, so $I = 1/(2 + 1) \approx 0.33$. Two modules rely on the envelope format, so the format should be extended compatibly rather than casually reshaped. Transaction Engine has $F_{in} = 1$ and $F_{out} = 3$, so $I = 3/(1 + 3) = 0.75$. It orchestrates lower-level modules, while few modules depend on its internals; therefore, its high-level API can evolve with less downstream impact.

The Lab 8 result states that the system decomposes into 13 modules in four layers, with 14 production dependencies and average coupling 1.08. The dependency gradient from volatile user-facing layers toward stable primitives is the quantitative argument for high cohesion and limited coupling in this particular graph.

9.12 Module-to-Code Traceability from Lab 8

Table 9.9: *Module-to-code traceability from Lab 8*

Diagram node	Code location	Evidence / responsibility cited
Transaction Engine	`internal/storage/engine.go`	Begin/Get/Put/Delete/Rollback/Commit operations.
Operational AI	`internal/ai/engine.go`	Declares the StatsProvider interface.
Routing Ring	`internal/shard/ring.go`	ShardFor with SHA-256 rendezvous scoring.
Wire Protocol	`internal/transport/protocol.go`	Envelope kinds, NotLeader, and LeaderHint.
Consensus	`internal/raft/`	State-machine apply interface, leadership transfer, and TimeoutNow.
Web Layer	`internal/httpapi/`	Adapter implements StatsProvider; REST, SSE, and dashboard.

Implementation Overview

10.1 Technology and Repository View

The supplied project materials describe a Go implementation and a package-oriented repository. The exact runtime configuration may vary by build, so deployment values should be taken from the checked-out repository and environment configuration rather than inferred from this report.

Conceptual request path: Client → HTTP API / Client Library → Shard Router → Raft Leader → WAL / Replication → MVCC Storage → Client Result. Metrics and AI diagnostics observe selected signals alongside this path.

10.2 Key Implementation Areas

1. **Transport and API:** request decoding, validation, routing, status-code mapping, and response serialization.
2. **Leader discovery:** follower response with leader hint, client-side hint caching, and retry against the leader.
3. **Replicated commit:** proposal creation, WAL append, follower replication, quorum acknowledgement, commit advancement, and application.
4. **Durable recovery:** replay committed log records and recover transaction-coordinator decisions as appropriate.
5. **MVCC transactions:** begin timestamp, snapshot reads, buffered writes, conflict check, and commit/abort outcome.
6. **Cross-shard coordination:** participant discovery, durable decision log, prepare, commit/abort, and restart recovery.
7. **Operational interfaces:** metrics, live events/dashboard, safe operational query handling, TLS validation, and optional AI diagnostics.

10.3 Error Handling Principles

- Return structured and meaningful errors for invalid input, missing keys, unsupported commands, untrusted certificates, and unavailable dependencies.
- Distinguish retryable leader-routing failures from terminal validation or transaction-conflict outcomes.
- Avoid reporting a final commit outcome before the relevant durability and consensus conditions have been satisfied.
- Treat observability and external AI failures as diagnostic failures, not as permission to bypass database rules.

Functional Test Strategy and Verification

11.1 Testing Objectives

The testing strategy checks functional correctness, concurrency control, durability, routing, security, operational query safety, metrics, and cross-shard atomicity. Test cases should be repeatable, linked to requirement identifiers, and assessed against observable expected results.

11.2 Test Levels

Table 11.1: *Test levels and representative examples*

Level	Purpose	Examples
Unit	Validate a module's local logic.	Key validation, MVCC visibility, timestamp ordering.
Integration	Validate interactions between modules.	HTTP-to-storage path, WAL replay, Raft apply.
End-to-end	Validate a user-visible workflow.	PUT/GET/DELETE, leader redirect, transaction commit.
Fault / recovery	Validate behaviour around failure and restart.	Crash after committed write, coordinator restart, participant failure.
Negative / security	Ensure unsafe or invalid actions are rejected.	Absent key, untrusted TLS certificate, unsupported operational query.

11.3 Test Register

Table 11.2: *Functional test register*

Test	Requireme	Scenario / input	Type	Expected result
TC-01	FR-032, FR-034	PUT a new key 'course' with value 'MCA', then GET it.	Positive	Value is stored and returned.
TC-02	SRS FR	GET a key that does not exist.	Negative	Not-found result is returned.
TC-03	FR-032, FR-066	DELETE an existing key, then GET it.	Positive	Key is absent after deletion.
TC-04	FR-066	DELETE a key that is already absent.	Negative	Defined absent-key behaviour is returned.
TC-05	FR-023–028	Commit a transaction with two writes to keys in the same shard.	Positive	Both writes commit consistently.

Test	Requiremer	Scenario / input	Type	Expected result
TC-06	FR-026–028	Two transactions concurrently update the same version.	Negative	Conflicting transaction aborts; committed value is preserved.
TC-07	FR-007–010	Restart a node after a committed write.	Positive	Committed value is recovered.
TC-08	FR-012, FR-002	Send a client request to a follower.	Positive	Leader hint/redirect allows retry.
TC-09	FR-003	Rejoin a follower after a committed write.	Positive	Follower catches up to committed state.
TC-10	FR-016	Request Prometheus-style metrics.	Positive	Metrics endpoint returns a valid response.
TC-11	FR-021	Submit an unrecognized natural-language query.	Negative	Unsupported command receives a safe response.
TC-12	FR-022	Connect using an untrusted TLS certificate.	Negative	Secure mode rejects the connection.
TC-13	FR-023	Send a valid REST PUT request.	Positive	Request is accepted and a valid response is returned.
TC-14	FR-073–078	Submit an atomic cross-shard transaction through the dedicated endpoint.	Positive	Coordinated transaction reaches one final outcome.
TC-15	FR-072	Submit a cross-shard request through the ordinary transaction endpoint.	Negative	Unsupported path is rejected clearly; no silent partial commit.

Status interpretation: “Implemented” in the source register is an implementation/coverage label. It does not mean each manual procedure was freshly run while preparing this consolidated report. The Lab 9 source itself notes that the repository’s full suite had passed previously, while a fresh rerun was blocked because the Go 1.25.0 toolchain was unavailable and downloading it was blocked in that environment.

11.4 Detailed Test Procedure: PUT and GET

Preconditions: a healthy cluster is running and the client can reach the API. Steps:

1. Send a valid PUT for key ‘course’ and value ‘MCA’.

2. Confirm the response indicates successful completion.
3. Send GET for key ‘course’.
4. Compare the returned value with ‘MCA’.
5. Repeat with an absent key to validate not-found behaviour.

Pass rule: the successful PUT is followed by a GET returning the stored value; absent keys return the documented not-found response without creating data.

11.5 Detailed Test Procedure: MVCC Conflict

Preconditions: two concurrent transactions can read/write the same key/version. Steps:

1. Begin transactions T1 and T2 before either commits.
2. Have both transactions read the same key/version.
3. Let T1 write and commit first.
4. Attempt to commit T2 with a conflicting write.
5. Read the key and inspect both transaction outcomes.

Pass rule: the first valid commit remains visible; the conflicting transaction is aborted or rejected; no partial conflicting write overwrites the winner.

11.6 Detailed Test Procedure: Crash and Restart Recovery

Preconditions: a test cluster with persistent data directory and a controllable node process. Steps:

1. Commit a known key–value write and confirm success.
2. Stop or terminate the node after the commit response.
3. Restart the node using the same persistent data directory.
4. Wait for recovery and cluster readiness.
5. Read the key through the API and compare the value.

Pass rule: the committed value is available after restart; no uncommitted partial state is presented as committed.

11.7 Detailed Test Procedure: Cross-Shard Atomicity

Preconditions: two or more shards and a test transaction whose keys route to different shards. Steps:

1. Submit a transaction with writes to keys assigned to at least two shards.
2. Force a participant prepare failure or a controlled coordinator interruption in a test environment.
3. Inspect the returned transaction outcome and the values on all involved shards.
4. Restart the coordinator if needed and allow recovery to finish.

Pass rule: the transaction resolves according to its durable decision; a failed prepare does not result in only a subset of the intended writes being committed.

11.8 Traceability Summary

Table 11.3: *Requirements-to-test traceability summary*

Requirement group	Representative IDs	Verification evidence
Key–value API	SRS FR 032–034, 066	TC-01 through TC-04 and TC-13; HTTP route tests.
Replication / durability	SRS FR 007–022, 083–084	TC-07 through TC-09; WAL, Raft, and restart tests.
Transactions / MVCC	SRS FR 023–034	TC-05 and TC-06; snapshot and conflict tests.
Observability / safe query	SRS FR 043–062, 067–069	TC-10 and TC-11; metrics and query-engine tests.
TLS / security	SRS FR 038–039, 063–065	TC-12; certificate-trust test.
Cross-shard transaction	SRS FR 070–079	TC-14 and TC-15; coordinator and gateway tests.

11.9 Implementation Evidence and Status Discipline

The supplied repository evidence identifies automated tests in the areas of REST key operations and metrics export, safe natural-language query handling, TLS trust validation, MVCC conflict behaviour, crash/restart durability, cross-shard atomicity, and gateway rejection of unsupported cross-shard cases. The project materials also state that a prior full test suite passed. A fresh rerun was previously blocked in the available environment because the required Go 1.25.0 toolchain download could not be completed.

Important interpretation: source code and existing test definitions demonstrate that checks are implemented in the repository, but do not prove that every test was freshly executed against the exact submitted build. For a formal release record, capture the commit hash, Go version, command, timestamp, and test output. The report therefore describes expected outcomes and known test coverage without fabricating a new pass result.

Laboratory Record: Experiment 9 – Functional Test Case Design

Functional Test Case Design

> **Aim:** To design functional test cases for kvstore and link each test case to its requirement.

Requirements to test

Req. ID	Requirement in simple words	SRS reference
R-01	Create, read and delete key-value entries; a missing key is reported as absent.	FR-032, FR-034, FR-066
R-02	Snapshot-isolated transactions; conflicting writes are rejected (first-committer-wins).	FR-023 to FR-028, FR-032, FR-033
R-03	Durable log, recovery after restart, majority commit and leader redirect.	FR-007 to FR-010, FR-015, FR-037, FR-040 to FR-042
R-04	REST API, metrics, safe natural-language queries, TLS checks and the cross-shard transaction boundary.	FR-038, FR-039, FR-043 to FR-048, FR-058 to FR-066, FR-072 to FR-079

Test case register

Test ID	Req.	Scenario	Expected result	Type
TC-01	R-01	PUT then GET a new key	PUT succeeds; GET returns the same value	Positive
TC-02	R-01	GET a missing key	Reported as not found; no value invented	Negative
TC-03	R-01	DELETE then GET	Delete succeeds; later GET shows the key absent	Positive
TC-04	R-01	Delete an absent key	Defined missing-key result; cluster stays healthy	Negative
TC-05	R-02	Commit two writes in one transaction (same shard)	Commit succeeds; both values visible	Positive
TC-06	R-02	Two transactions write the same key	First commit wins; the second is rejected	Negative
TC-07	R-03	Restart after a committed write	Committed value is still available	Positive
TC-08	R-03	Request sent to a follower	Client is redirected to the leader and gets the result	Positive
TC-09	R-04	Request Prometheus metrics	Metric samples in Prometheus text format	Positive
TC-10	R-04	Unrecognised natural-language query	Safe help reply; no command executed	Negative
TC-11	R-04	Connect with an untrusted TLS certificate	Connection is rejected	Negative
TC-12	R-04	Valid REST PUT request	Success reply; a later read returns the value	Positive
TC-13	R-04	Cross-shard transaction	/api/v1/xtxn coordinates it; /api/v1/txn rejects it	Positive / Negative

Result

In this experiment we wrote 13 positive and negative test cases for kvstore and linked each one to a requirement.

Detailed Test Register and Execution Detail

Expanded Lab 9 Test Register and Exe- cution Detail

This appendix preserves the test-design terminology and requirement IDs used in the original Lab 9 artefact. The Lab 9 shorthand FR-02, FR-03, etc. is a local test-assignment namespace and is not the same numbering scheme as the formal SRS IDs labelled SRS FR 001–084 earlier in this report. They are intentionally kept separate to avoid false one-to-one mappings.

11.10 Selected Lab 9 Requirements

The selected requirements below connect the laboratory's functional test design to observable scenarios. The status wording is retained as recorded in the source material.

Table 11.4: *Selected Lab 9 requirements and associated test scenarios*

Test ID	Requirement	Scenario / input	Type	Source status
TC-01	FR-032, FR-034	Create a key and read it back.	Positive	Implemented
TC-02	SRS FR	Read a missing key.	Negative	Implemented
TC-03	FR-032, FR-066	Delete an existing key, then read it.	Positive	Implemented
TC-04	FR-066	Delete an absent key.	Negative	Implemented
TC-05	FR-023–028	Commit two same-shard writes in one transaction.	Positive	Implemented
TC-06	FR-026–028	Concurrently update the same version.	Negative	Implemented
TC-07	FR-007–010	Restart after a committed write.	Positive	Implemented
TC-08	FR-012, FR-002	Send request through a follower.	Positive	Implemented
TC-09	FR-003	Rejoin a follower after a committed write.	Positive	Implemented
TC-10	FR-016	Request Prometheus metrics.	Positive	Implemented
TC-11	FR-021	Submit an unsupported operational query.	Negative	Implemented
TC-12	FR-022	Connect with an untrusted TLS certificate.	Negative	Implemented
TC-13	FR-023	Send a valid REST PUT.	Positive	Implemented
TC-14	FR-073–078	Submit an atomic cross-shard transaction through the dedicated endpoint.	Positive	Implemented
TC-15	FR-072	Submit a cross-shard request through the ordinary endpoint.	Negative	Implemented

11.11 Original Register A1: Scenario and Classification

The source register classifies scenarios by expected outcome and records an implementation status. The register is presented in a readable multi-page table; requirement identifiers are kept as they appear in the supplied lab material.

Table 11.5: *Original Register A1: scenario and classification*

ID	Req. ID	Scenario	Class	Status
A1-01	FR-032, FR-034	PUT a new key and read it back.	Positive	Implemented
A1-02	FR-032	GET a key that does not exist.	Negative	Implemented
A1-03	FR-032, FR-066	DELETE a key and verify it is absent.	Positive	Implemented
A1-04	FR-066	DELETE an already absent key.	Negative	Implemented
A1-05	FR-023–028	Commit multiple same-shard writes atomically.	Positive	Implemented
A1-06	FR-026–028	Cause a concurrent write conflict.	Negative	Implemented
A1-07	FR-007–010	Restart after a committed write.	Positive	Implemented
A1-08	FR-012, FR-002	Route a request through a follower.	Positive	Implemented
A1-09	FR-003	Rejoin a follower and verify catch-up.	Positive	Implemented
A1-10	FR-016	Request operational metrics.	Positive	Implemented
A1-11	FR-021	Submit an unsupported operational query.	Negative	Implemented
A1-12	FR-022	Present an untrusted TLS certificate.	Negative	Implemented
A1-13	FR-023	Send a valid REST PUT request.	Positive	Implemented
A1-14	FR-073–078	Use the dedicated cross-shard transaction endpoint.	Positive	Implemented
A1-15	FR-072	Use the ordinary endpoint for a cross-shard transaction.	Negative	Implemented

11.12 Test Procedure Details and Pass Rules

11.12.1 TC-01: Create a key and read it back

Precondition: healthy cluster and a client endpoint are available. Input: key course, value MCA. Send PUT, check the success response, send GET for the same key, and compare the returned value. Pass rule: GET returns exactly MCA after successful PUT.

11.12.2 TC-06: Conflicting MVCC writes

Begin two transactions before either commits. Have both read the same key/version, let T1 write and commit, then attempt T2's conflicting commit. Inspect the outcomes and final value. Pass rule: the first valid commit remains visible; the losing conflicting transaction is aborted/rejected and does not overwrite the winner or apply a partial conflicting write.

11.12.3 TC-07: Crash and restart recovery

Commit a known write, stop the node after the commit response, restart it with the same persistent data directory, wait for readiness, then read the key. Pass rule: the committed value is recovered; incomplete or uncommitted state is not presented as committed.

11.12.4 TC-08: Request through a follower

Send a client operation to a follower rather than the known leader. Check that the reply identifies the non-leader condition and supplies a leader hint when known; allow the client library to retry. Pass rule: the operation reaches the leader and returns the correct result without applying the write twice.

11.12.5 TC-10: Prometheus metrics

Request the metrics endpoint and inspect the response content type and payload. Pass rule: the response is valid Prometheus text and includes the expected metric names/values for the running test process.

11.12.6 TC-11: Unsupported operational query

Submit a natural-language request that is not in the allowed command set. Pass rule: the system returns a help/unsupported response and does not execute an arbitrary operation.

11.12.7 TC-12: Untrusted TLS certificate

Enable the configured secure mode and connect using a certificate not trusted by the configured authority. Pass rule: the handshake/connection is rejected; the test must not disable verification to force a success.

11.12.8 TC-14 and TC-15: Cross-shard transaction boundary

For TC-14, use the dedicated cross-shard endpoint with operations routed to distinct shards and inspect the coordinator outcome and values on all participants. For TC-15, submit the same kind of cross-shard request to the ordinary transaction endpoint. Pass rules: the dedicated endpoint follows the supported atomic protocol; the ordinary endpoint rejects unsupported multi-shard operations before partial mutation.

11.13 Repository Test Evidence Named in Lab 9

The source archive and report name automated tests that correspond to the following areas. These names are evidence of repository test coverage; the report does not claim a new run unless a fresh execution log is available.

- REST key operations and metrics export: TestHTTPPKVRoundTrip, TestHTTPMetricsPromAndJSON.
- Safe natural-language query handling: TestNLQueryEngineUnrecognizedFallsBackToHelp.
- TLS trust validation: TestClientRejectsUntrustedServerWithoutInsecureSkip.
- MVCC conflict and durable restart recovery: TestSnapshotIsolationWriteConflictFirstCommitterWins,

TestCrashAndRestartPreservesCommittedWrites.

- Cross-shard atomic commit and ordinary-endpoint rejection: TestCrossShardTxnCommitsAtomicallyOnAllSha

TestGatewayRejectsCrossShardTxn.

11.14 Traceability and Execution Record

Table 11.6: *Test evidence to retain for final execution record*

Test group	Requirement coverage	Evidence to retain
CRUD and REST	FR-04–FR-06, FR-23	Request/response payloads, status code, final value, and test output.
Transactions and MVCC	FR-07, FR-08	Transaction IDs, commit/abort results, final values, and concurrency test output.
Durability and replication	FR-03, FR-10	Restart logs, recovered value, and follower catch-up output.
Leader redirect	FR-02, FR-12	Initial follower address, leader hint, and retry result.
Metrics and operational query	FR-16, FR-21	Metrics response and safe fallback response.
TLS and gateway boundary	FR-22, FR-28, FR-72–FR-78	TLS handshake result, coordinator/participant results, and proof of no partial mutation.

A formal execution record should include the exact source revision/commit, Go toolchain version, command executed, execution timestamp, pass/fail/blocked status, and saved output. Where a test has only been designed or its implementation has been identified, its status should not be changed to “Pass” without that evidence.

Security, Reliability, and Operational Considerations

12.1 Security

- Validate TLS certificates in secure mode and reject untrusted server certificates.
- Keep administrative operations behind controlled interfaces and avoid exposing privileged actions

through free-form natural-language commands.

- Bound the diagnostic evidence sent to external AI providers and avoid sending secrets or unnecessary

data.

- Treat LLM responses as untrusted suggestions; they must not directly mutate cluster state or

override transaction outcomes.

12.2 Reliability and Recovery

- Use the WAL and Raft commit position as the basis for committed-state recovery.
- Recover transaction coordinator state from durable decision records.
- Ensure replicas apply committed entries deterministically.
- Handle transient transport failures through bounded retries and explicit error responses.
- Keep the core data path operational when the observability or external LLM path is unavailable,

subject to deployment configuration.

12.3 Operational Monitoring

The operator should be able to inspect request counts, latency distributions, error rates, cluster membership, leader state, replication progress, and transaction outcomes where the implementation exposes them. Metrics names and labels should be taken from the running build's metrics endpoint. AI- generated hypotheses should be displayed alongside the evidence and should be labelled as hypotheses, not confirmed causes.

Limitations and Future Enhancements

13.1 Current Documentation Limitations

- Performance and scale claims are not stated as measured results because no reproducible benchmark

report was included as a source for this consolidated document.

- Production-grade operational targets, backup/restore policy, data-retention policy, and deployment-

specific capacity limits require explicit values.

- Exact API schemas and command-line flags should be checked against the repository version used

for the final demonstration.

- Some design artefacts use conceptual package labels; the concrete source tree should be used to

confirm the current implementation names before a production release.

13.2 Recommended Enhancements

1. Add a reproducible benchmark suite for throughput, tail latency, and cluster scaling.
2. Add fault-injection automation for leader changes, disk errors, network partitions, and coordinator

crashes.

1. Add explicit operational runbooks for backup, restore, certificate rotation, and incident response.
2. Add a versioned OpenAPI specification and generate client examples from the same source.
3. Add structured audit events for operator actions and AI diagnostic requests.
4. Add retention and cleanup policies for WAL segments and MVCC versions, with tests for safe

compaction.

1. Capture CI artefacts including test reports, race detector results, coverage summaries, and build

provenance.

Conclusion

The KVStore project brings together the core ideas of distributed storage, transaction processing, fault tolerance, and operational visibility in one software-engineering case study. The requirements specification defines expected behaviour and constraints; the selected risk-driven, iterative process provides a disciplined way to implement high-risk capabilities in increments; and the use-case, DFD, UML, and sequence views describe system boundaries, information movement, structure, and runtime collaboration.

The design emphasises correctness at the points where distributed systems are most vulnerable. Durable replicated commitment protects accepted writes; deterministic state-machine application keeps replicas aligned; MVCC and first-committer-wins checks prevent conflicting transactions from silently overwriting committed data; and the two-phase commit coordinator uses a durable decision record to recover cross-shard outcomes. The modular design separates transport, routing, consensus, persistence, versioning, metrics, security configuration, and advisory AI diagnostics so that a failure or change in one concern does not unnecessarily compromise the others.

The laboratory work also demonstrates that a design is only as convincing as its evidence. Requirements need traceable test cases, recovery and failure paths need explicit validation, and implementation-status labels should not be confused with a fresh test execution. Before release or final submission, the team should run the full test suite against the exact submitted revision and retain reproducible build, test, race-detection, and recovery evidence.

Thank you

Group 1 | CS545 Software Engineering | IIIT Vadodara

Consolidation Map: Laboratory Artefacts

Table A.1: *Consolidation map of laboratory artefacts*

Lab / tutorial	Artefact incorporated	Where represented in this report
Lab 1	Software Requirements Specification	Chapter 3: functional and non-functional requirements; formal SRS requirement index.
Lab 2	Process-model selection	Chapter 4: process model, development increments, and risk register.
Lab 3	Use-case diagram	Chapter 5: actors and principal use cases.
Lab 4	Context, Level 1, and Level 2 DFDs	Chapters 2 and 6: system context, subsystem flows, and package decomposition.
Tutorial 5	UML class diagram	Chapter 7: class responsibilities and structural principles.
Tutorial 6	Three sequence diagrams	Chapter 8: leader redirect, snapshot isolation, and cross-shard 2PC.
Lab 7	No separate Lab 7 source artefact was identifiable in the supplied files	Related architecture, implementation, and verification topics are covered in Chapters 9–12. If a specific Lab 7 handout or required deliverable exists, it should be added rather than inferred.
Lab 8	Modular design report and module architecture	Chapter 9: module responsibilities, interfaces, cohesion, and coupling.
Lab 9	Functional test design and student submission	Chapter 11: test strategy, test register, detailed procedures, traceability, and evidence status.

Quick Reference: Key Terms

Table B.1: *Key terms used in the kvstore report*

Term	Meaning in kvstore
Raft	A consensus protocol used to elect a leader and replicate an ordered log.
WAL (Write-Ahead Log)	Durable record of operations used for recovery and replicated state application.
HLC (Hybrid Logical Clock)	Timestamp mechanism combining logical ordering with physical time information.
MVCC	Multi-version concurrency control; maintains versions and supports snapshot reads/conflict validation.
Snapshot isolation	Transaction reads from a consistent view associated with its begin timestamp.
First-committer-wins	When transactions conflict, the transaction that loses commit validation is aborted rather than overwriting the winner.
Shard	A partition of key space; each key routes to its owning shard.
Two-phase commit	Coordinator protocol with prepare and commit/abort phases for a transaction involving multiple shards.
Leader redirect	A follower response that tells the client where to retry a leader-only operation.
Observability	Metrics, events, and diagnostics that help operators understand system behaviour.

Team and Submission Information

Table C.1: *Team and submission information*

Role	Name	Enrollment ID
Team Leader	Neha Nupur	20251651064
Member	Aditi Garg	20251651008
Member	Shivam Kumar Soni	20251651084

References

[1] IIIT Vadodara, *CS545 Software Engineering Laboratory Handouts and Tutorials*, MCA Programme, Indian Institute of Information Technology Vadodara.

[2] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.

[3] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proceedings of the USENIX Annual Technical Conference (USENIX ATC)*, 2014, pp. 305–319.

[4] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, and P. O’Neil, “A Critique of ANSI SQL Isolation Levels,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1995, pp. 1–10.

[5] D. Thaler and C. Ravishankar, “A Name-Based Mapping Scheme for Rendezvous,” in *Proceedings of the IEEE International Conference on Distributed Computing Systems*, 1998.

[6] J. Gray and L. Lamport, “Consensus on Transaction Commit,” *ACM Transactions on Database Systems*, vol. 31, no. 1, pp. 133–160, 2006.

[7] Group 1, *KVStore Software Requirements Specification (SRS), README, and Project Documentation*, CS545 Software Engineering, IIIT Vadodara, project materials supplied with this report.

Thank You

for reading our report

GROUP 1 | CS545 SOFTWARE ENGINEERING

Indian Institute of Information Technology Vadodara